アナログ開発ツールの使い方 (SKY130)

森　瑞紀 (Mizuki MORI) , 3zki@ishi-kai.org

今村　謙之 (Noritsuna IMAMURA) , noritsuna@ishi-kai.org

Rev.1

アジェンダ

- アナログLSIの設計フロー
- 開発ツールの紹介、PDKの中身
- アナログLSI設計デモンストレーション　（CMOSインバータ）

アナログLSIの設計フロー

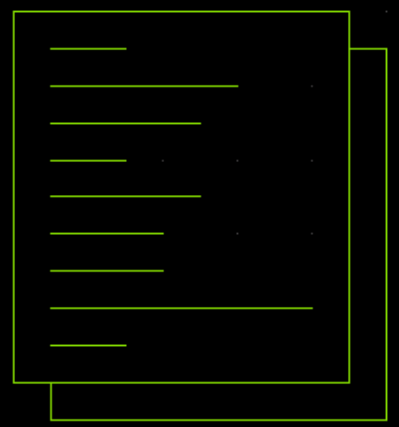
どのような設計をするか、どのような開発ツールを使用するか

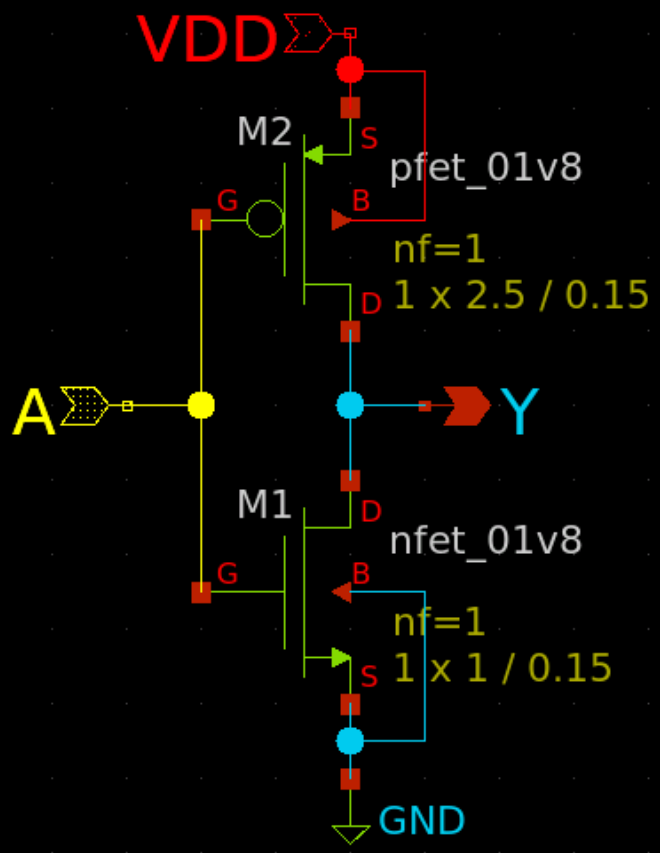
アナログLSIの設計フロー

1. 回路図（ベンチマーク）を描く
2. シミュレーションをする
3. 回路図を基にレイアウトを描く
4. レイアウトを検証する
5. レイアウトを基に寄生成分を考慮したシミュレーションをする

6. （フレームに載せる）

回路図(ベンチマーク)を描いて・・・

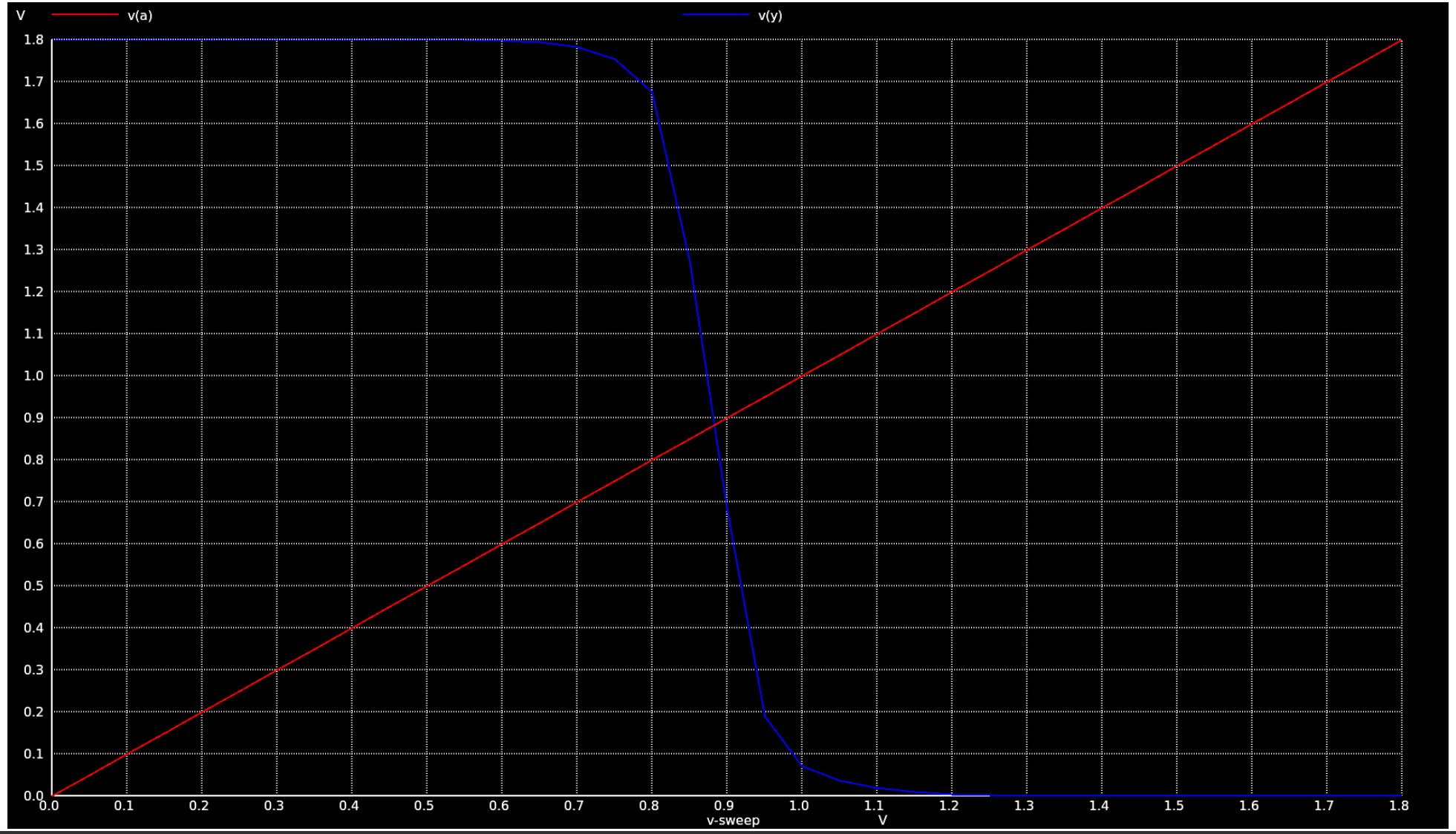




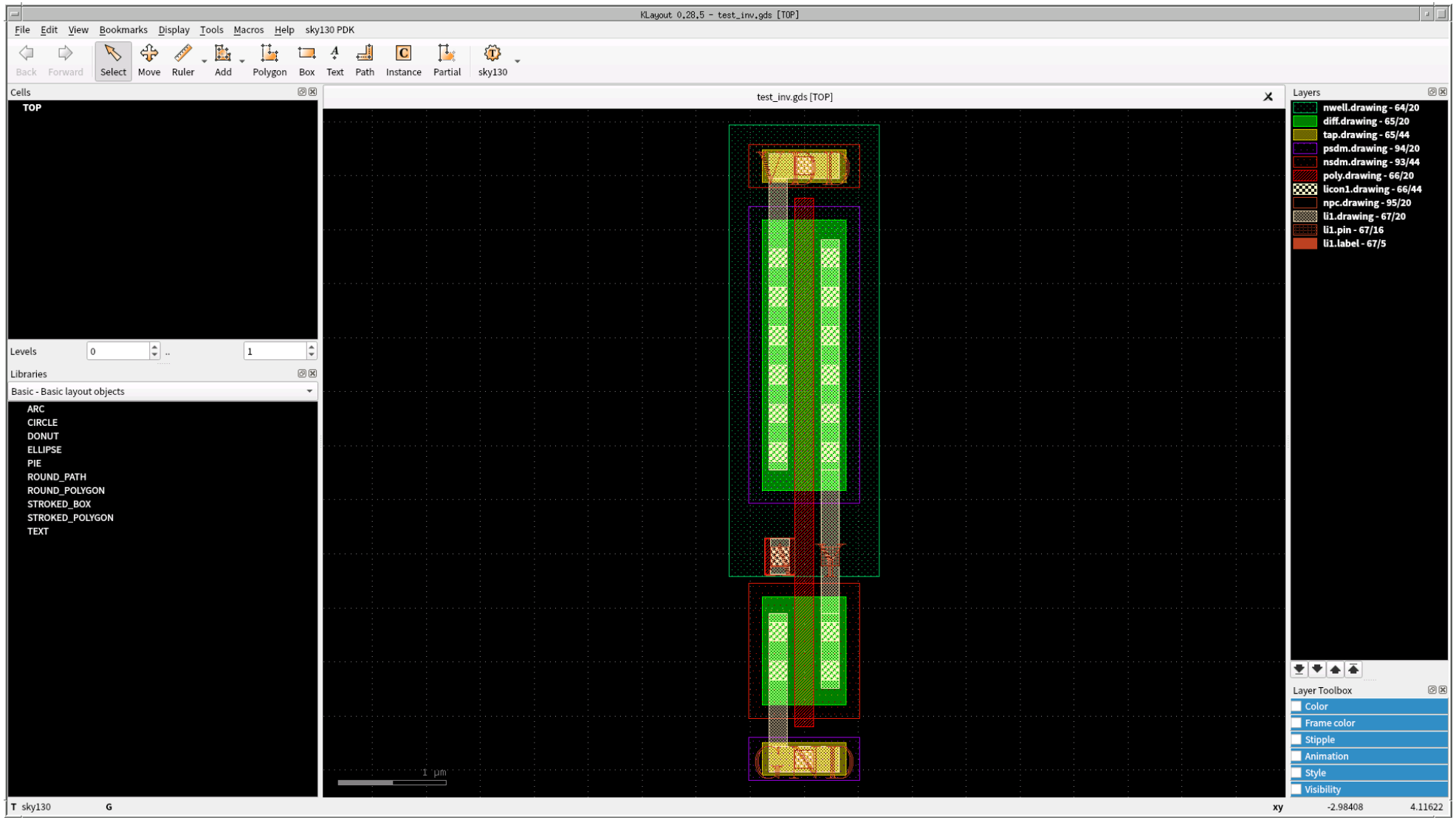
COMMANDS
SIM=ngspice

```
VD VDD 0 dc 1.8
VA A 0 dc 0
.control
save all
dc VA 0 1.8 0.05
plot v(a) v(y)
.endc
```

論理検証をして・・・

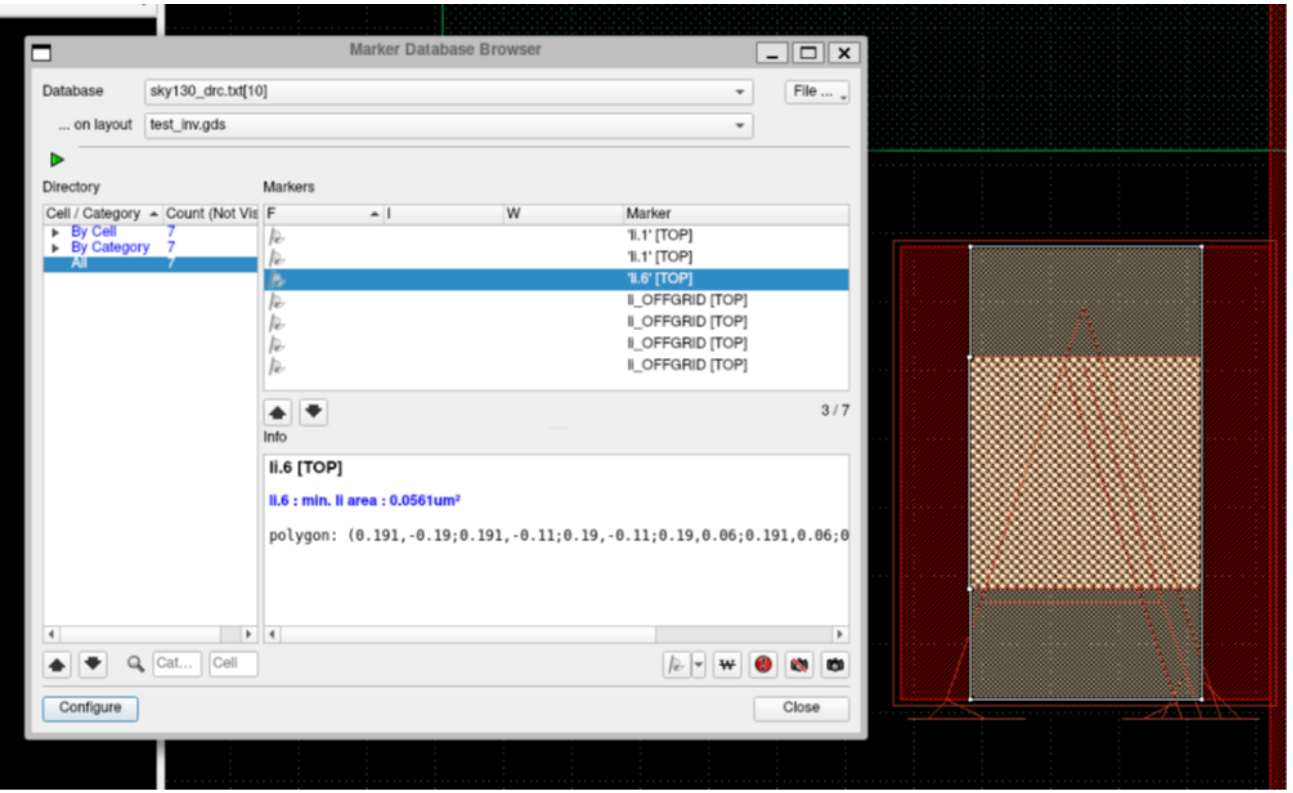

V
v(a)
v(y)
1.8
1.7
1.6
1.5
1.4
1.3
1.2
1.1
1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1.0
1.1
1.2
1.3
1.4
1.5
1.6
1.7
1.8
v-sweep
V

レイアウトを描いて・・・

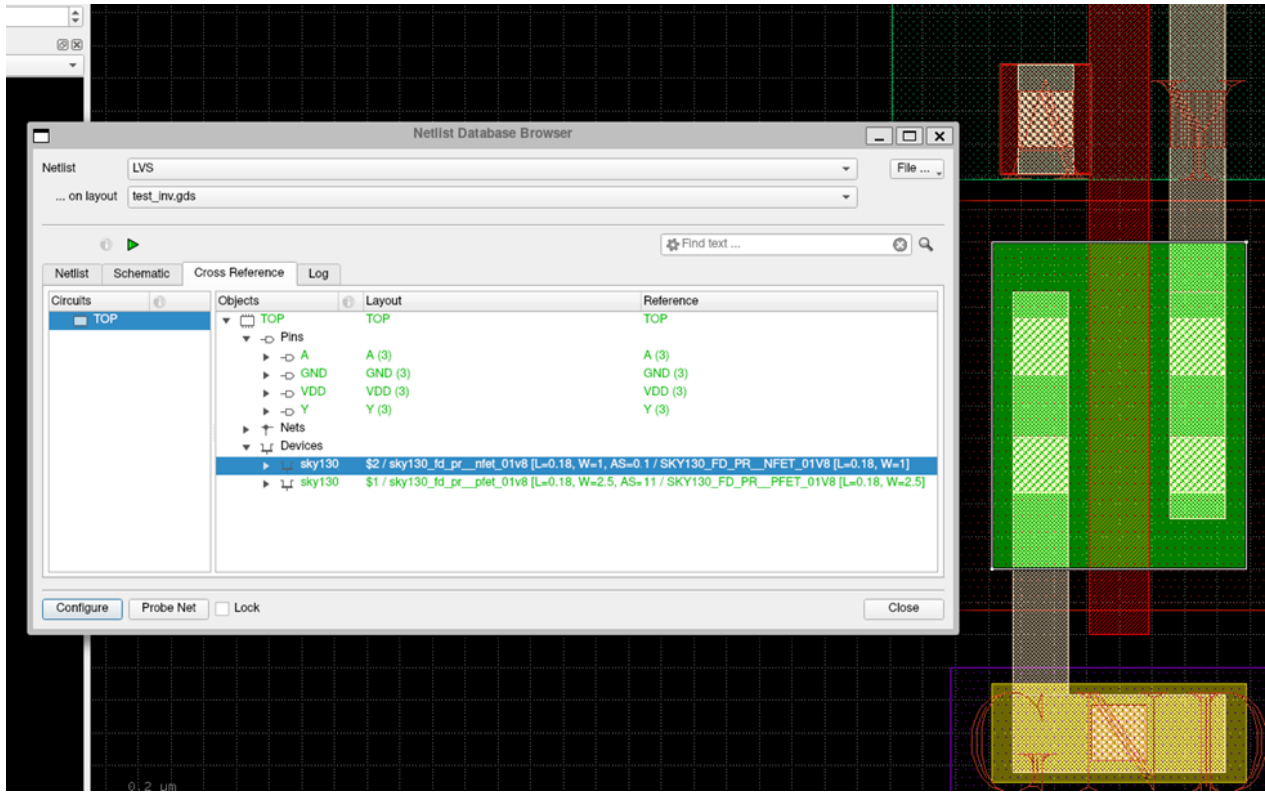

KLayout 0.28.5 - test_inv.gds [TOP]
File Edit View Bookmarks Display Tools Macros Help sky130 PDK
Back Forward Select Move Ruler Add Polygon Box Text Path Instance Partial sky130
Cells
TOP
Levels 0 .. 1
Libraries
Basic - Basic layout objects
ARC
CIRCLE
DONUT
ELLIPSE
PIE
ROUND_PATH
ROUND_POLYGON
STROKED_BOX
STROKED_POLYGON
TEXT
test_inv.gds [TOP]
1 µm
Layers
nwell.drawing - 64/20
diff.drawing - 65/20
tap.drawing - 65/44
psdm.drawing - 94/20
nsdm.drawing - 93/44
poly.drawing - 66/20
licon1.drawing - 66/44
npc.drawing - 95/20
li1.drawing - 67/20
li1.pin - 67/16
li1.label - 67/5
Layer Toolbox
Color
Frame color
Stipple
Animation
Style
Visibility
T sky130 G
xy -2.98408 4.11622

レイアウトを検証して・・・

DRC



LVS



寄生成分を抽出して・・・

Magic-PEX

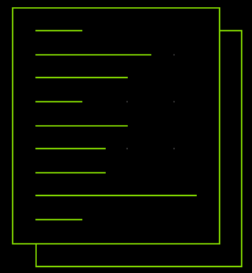
File Edit View Search Terminal Help

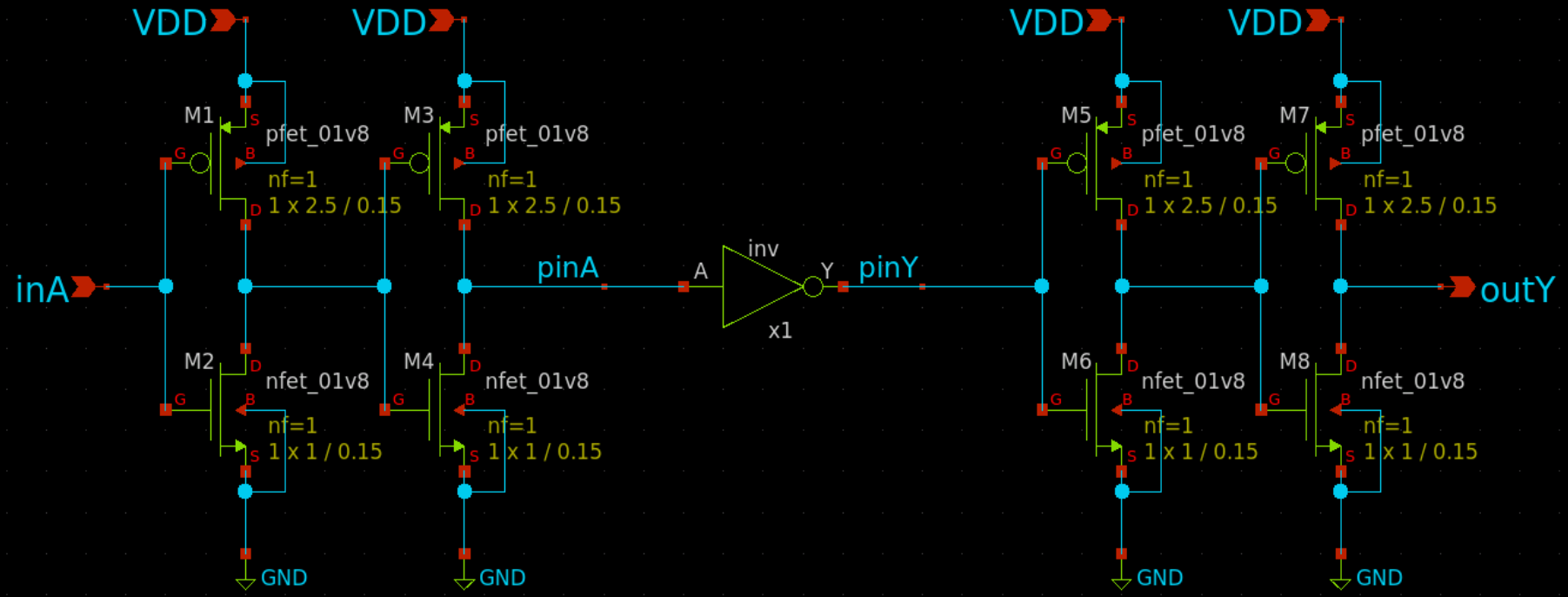
```
Loading sky130A Device Generator Menu ...
Loading "/home/user/.klayout/macros/sky130_magic_pex.tcl" from command line.
Warning: Calma reading is not undoable!  I hope that's OK.
Library written using GDS-II Release 6.0
Library name: LIB
Reading "TOP".
CIF file read warning: CIF style sky130(): units rescaled by factor of 5 / 1
Extracting TOP into TOP.ext:
exttosim finished.
exttospice finished.
exttospice finished.
* NGSPICE file created from TOP.ext - technology: sky130A

.subckt TOP A Y VDD GND
X0 Y A VDD VDD sky130_fd_pr__pfet_01v8 ad=7.5e+11p pd=5.6e+06u as=7.5e+11p ps=5.
6e+06u w=2.5e+06u l=180000u
X1 Y A GND GND sky130_fd_pr__nfet_01v8 ad=3e+11p pd=2.6e+06u as=3e+11p ps=2.6e+0
6u w=1e+06u l=180000u
C0 A Y 0.05fF
C1 VDD Y 0.17fF
C2 A VDD 0.18fF
.ends
```

ベンチマークを作成して…

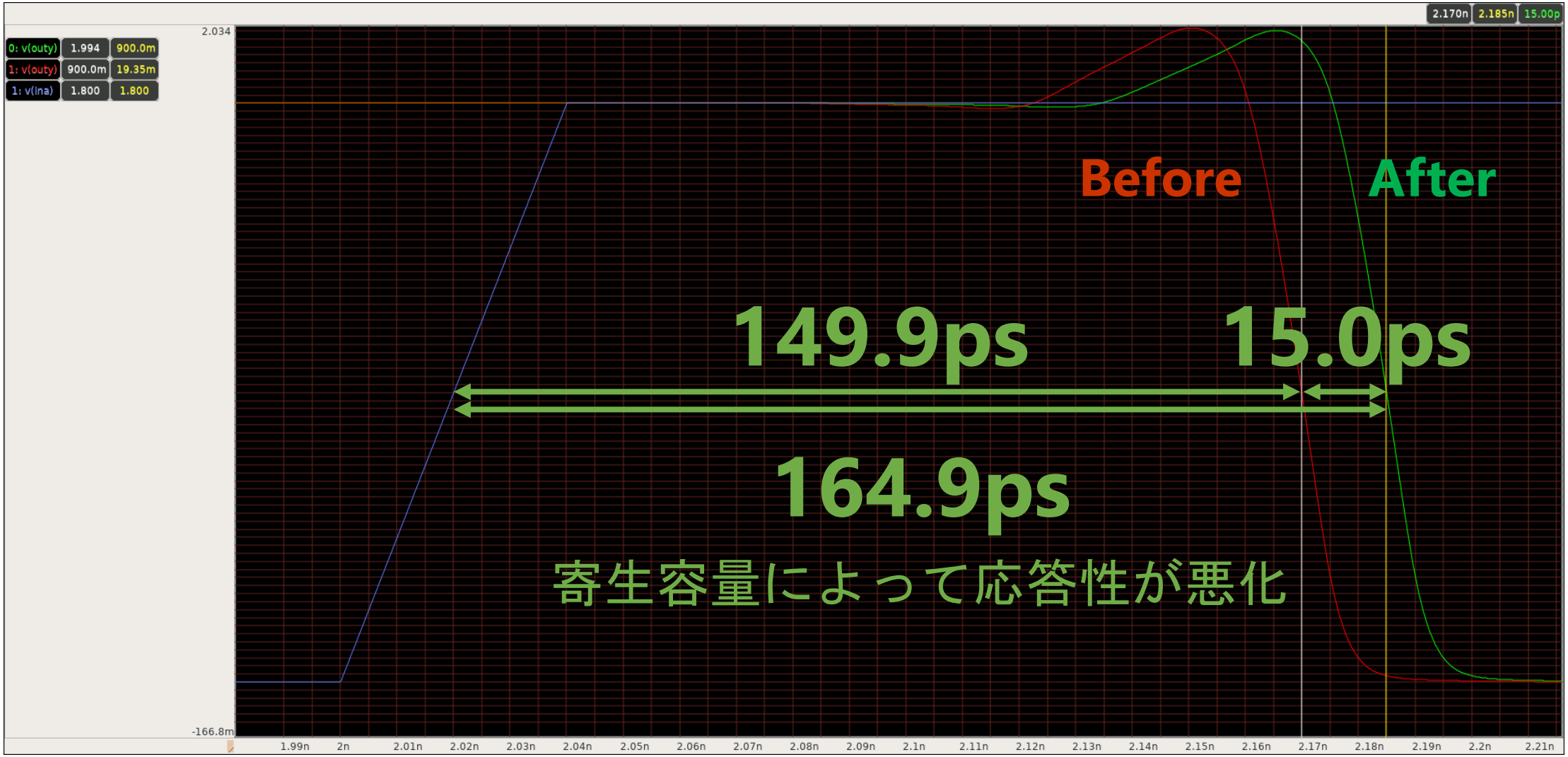
ngspice

```
VA inA 0 pulse(0 1.8 0 40p 40p 1n 2n) dc 0
VD VDD 0 dc 1.8
.include ~/TOP_pex_extracted.spice
.control
tran 1p 4n
wrdata ~/inv_bench.txt v(ina) v(outy)
write ~/inv_test_pex.raw
.endc
```





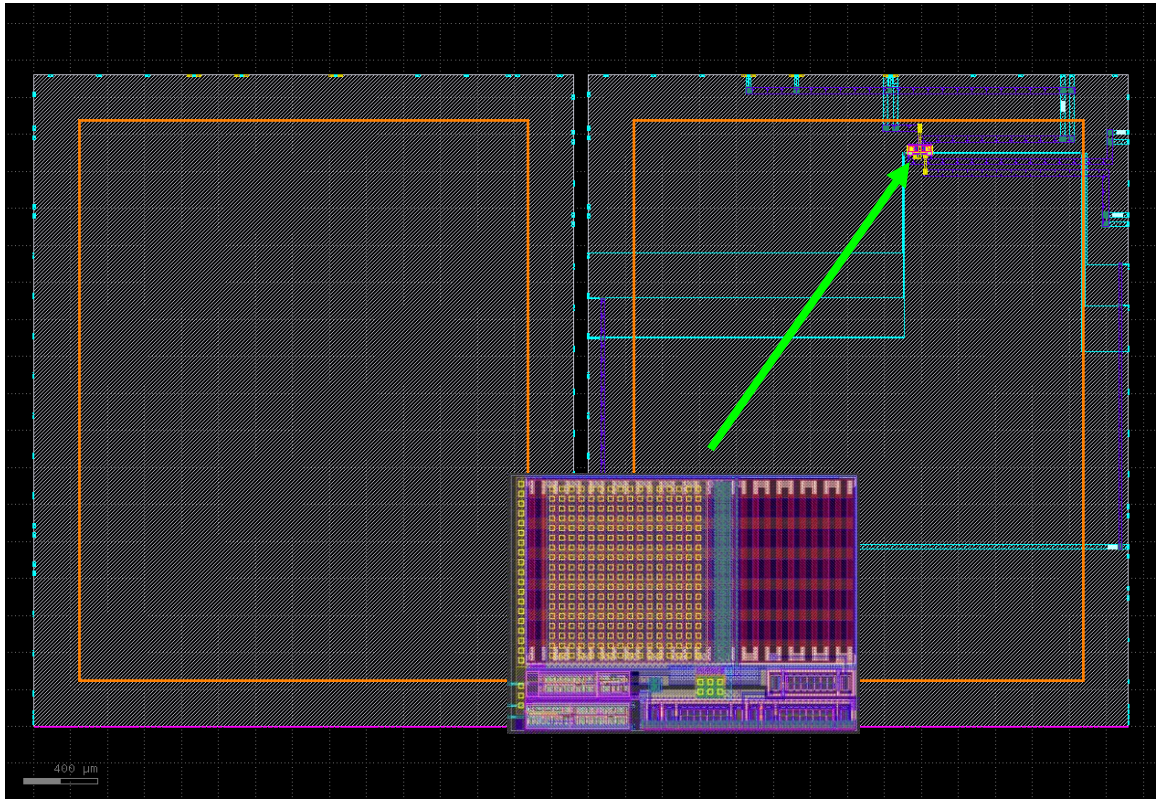
ポストレイアウトシミュレーションをする

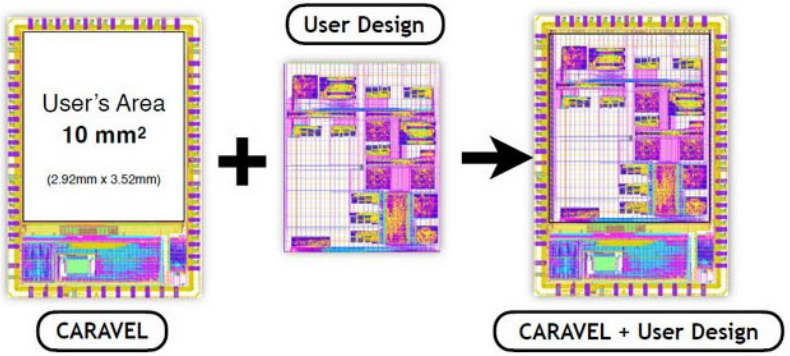


全て完成したら指定のフレームに回路を載せる

例: Caravel harness (SKY130)

- 2.92 mm × 3.52mm user's area
- 38 IO ports
- 4 Power ports
- 128 Logic analyzer probes





そもそもLSI回路設計に必要なものとは？

- **プロセスデザインキット　PDK**
  - **シミュレーション用モデルライブラリ (SPICE)**
  - **検証ツール用ルールファイル (DRC, LVSなど)**
  - **スタンダードセルライブラリ**
    - **レイアウト (GDS)**
    - **ネットリスト(SPICE)**
    - 自動配線ルール(LEF)
    - Verilogシミュレーションライブラリ (LIB, V)
  - **一部指定されたレイアウト (BJT)**

- PDKで指定された各種ツール　（EDAツール）
  - **回路図エディタ** or Verilogコンパイラ
    - **回路図エディタ：xschem**
  - **レイアウトエディタ** or 自動レイアウトツール
    - **レイアウトエディタ：klayout**, magic
  - **SPICEシミュレータ** or Verilogシミュレータ
    - **SPICEシミュレータ：ngspice,** xyce /zīs/
  - **検証ツール**
    - **検証ツール：klayout, magic, netgen**

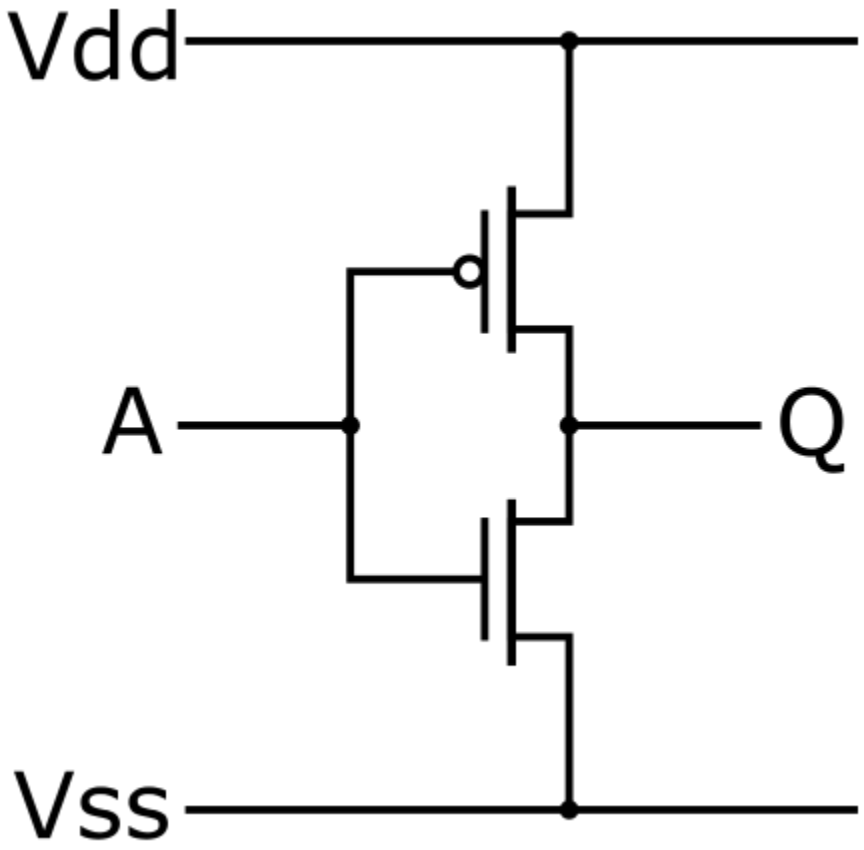
ローカルでのセットアップ方法

- セットアップツール
  - https://github.com/ishi-kai/OpenEDA-PDK_SetupScript
    - bash eda-setup.sh
    - bash pdk_sky130-setup.sh

アナログLSI設計デモンストレーション

CMOSインバータ作成

CMOSインバータ

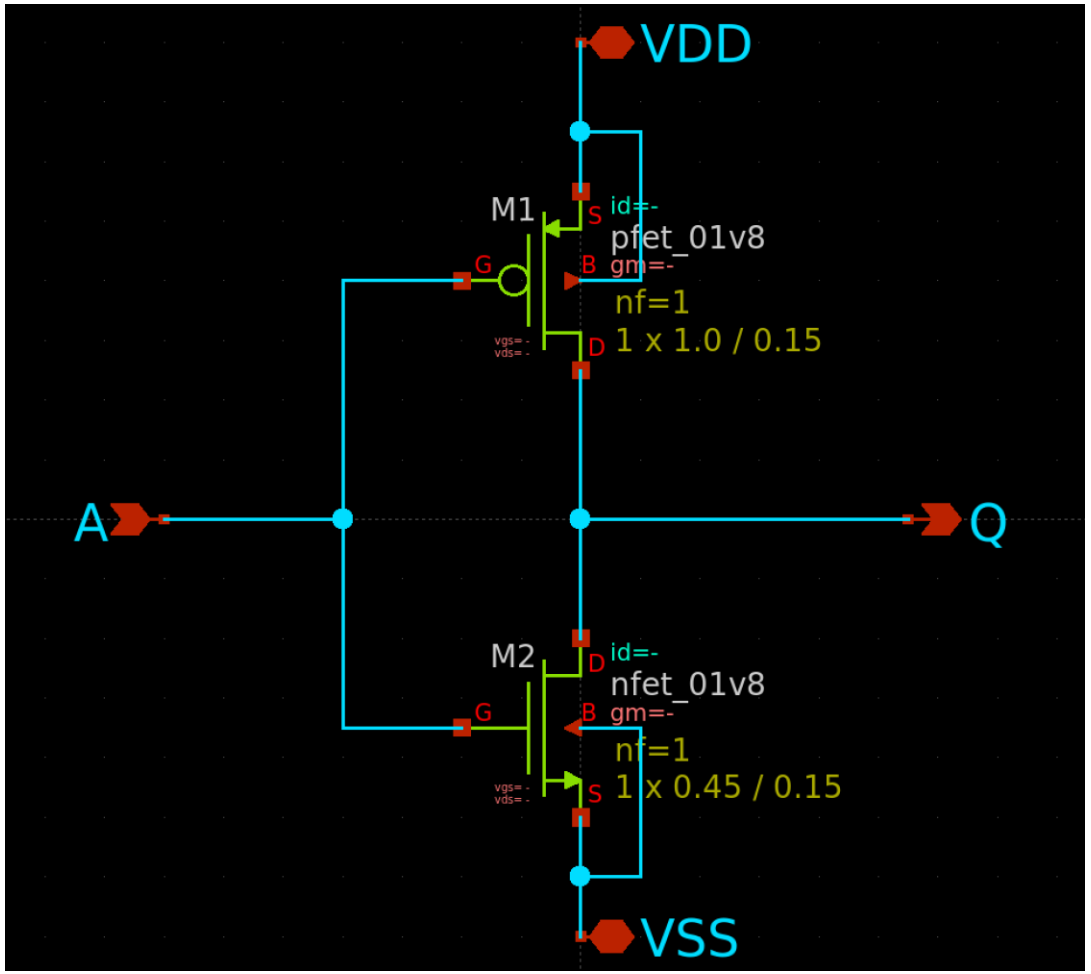


入力A	出力Q
L	H
H	L

入力A	出力Q
Vss	Vdd
Vdd	Vss

回路図の例

- N-FET, P-FET
  - 対応電圧ごとに複数ある



アナログLSIの設計フロー

1. **回路図（ベンチマーク）を描く**
2. **シミュレーションをする**
3. 回路図を基にレイアウトを描く
4. レイアウトを検証する
5. レイアウトを基に寄生成分を考慮したシミュレーションをする

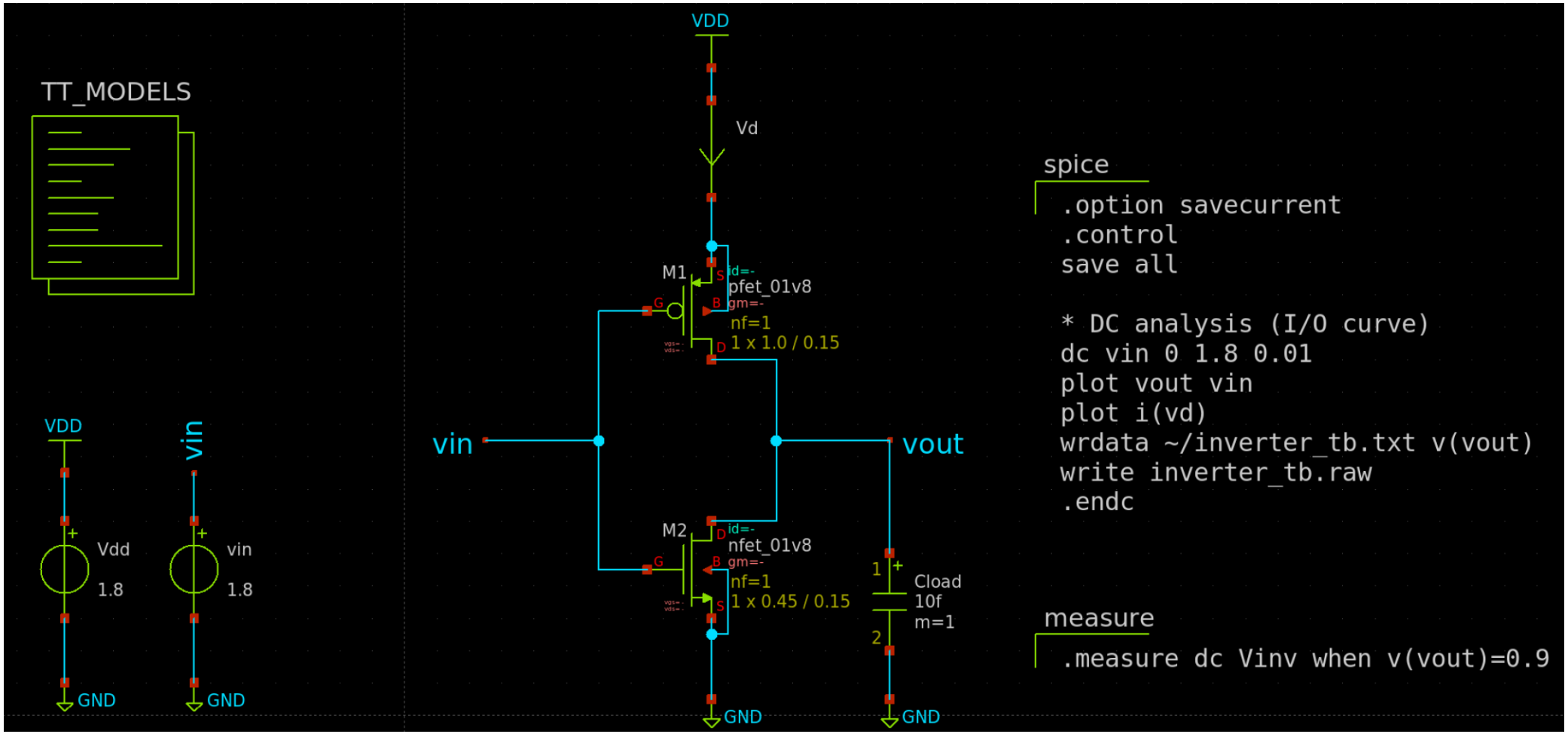
6. （フレームに載せる）

論理検証

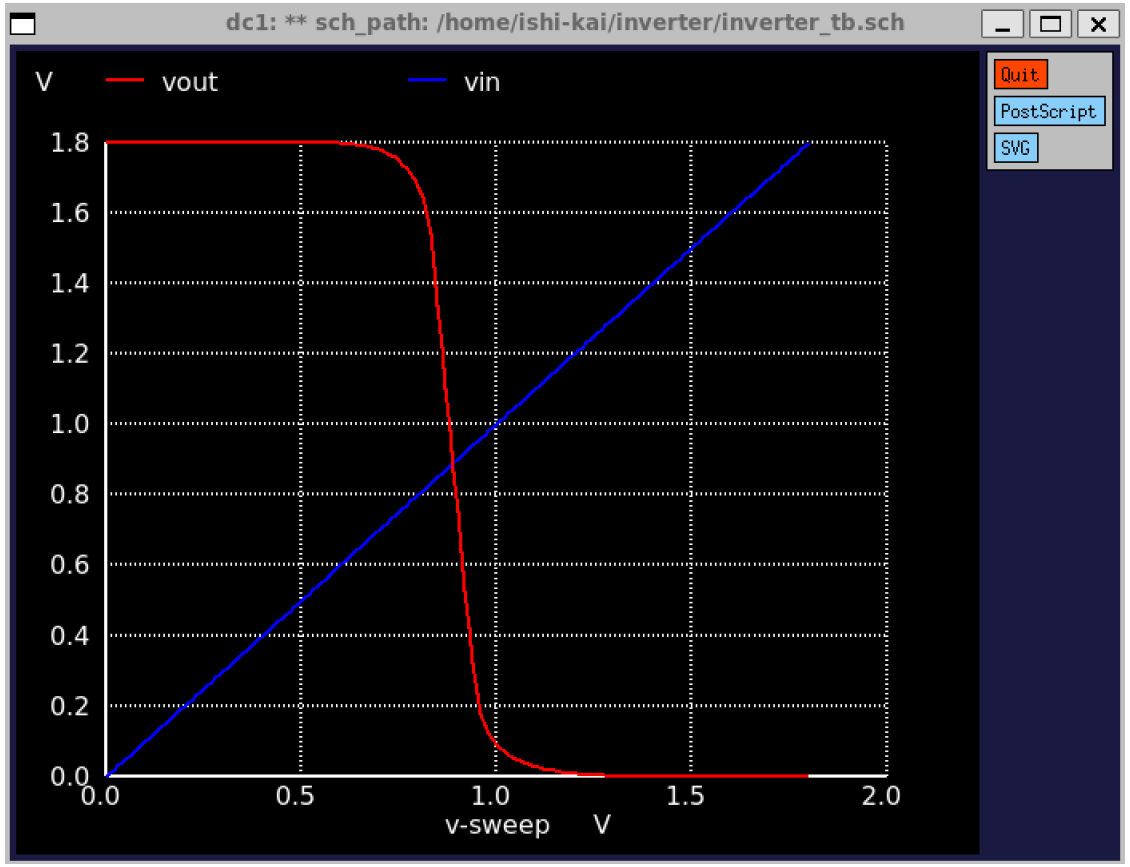
- インバータでは入力に対して反転した出力が確認できれば良い
  - 入力 0 V → 出力 Vdd V, 入力 Vdd V → 出力 0 V
  - DC解析でCMOSインバータの動作を見てみる

- DC解析　　電圧を変動させたときの定常応答
- tran解析　時間を変動させたときの過渡応答
- AC解析　　周波数を変動させたときの定常応答

ベンチマーク例


TT_MODELS
VDD
Vd
M1
pfet_01v8
nf=1
1 x 1.0 / 0.15
vin
vout
M2
nfet_01v8
nf=1
1 x 0.45 / 0.15
Cload
10f
m=1
VDD
Vdd
1.8
vin
1.8
GND
GND
GND
GND
spice
.option savecurrent
.control
save all
* DC analysis (I/O curve)
dc vin 0 1.8 0.01
plot vout vin
plot i(vd)
wrdata ~/inverter_tb.txt v(vout)
write inverter_tb.raw
.endc
measure
.measure dc Vinv when v(vout)=0.9

シミュレーション結果



入力A = 0 V　　→出力Q = 3.3 V

入力A = 3.3 V　　→出力Q = 0 V

- インバータとして機能している

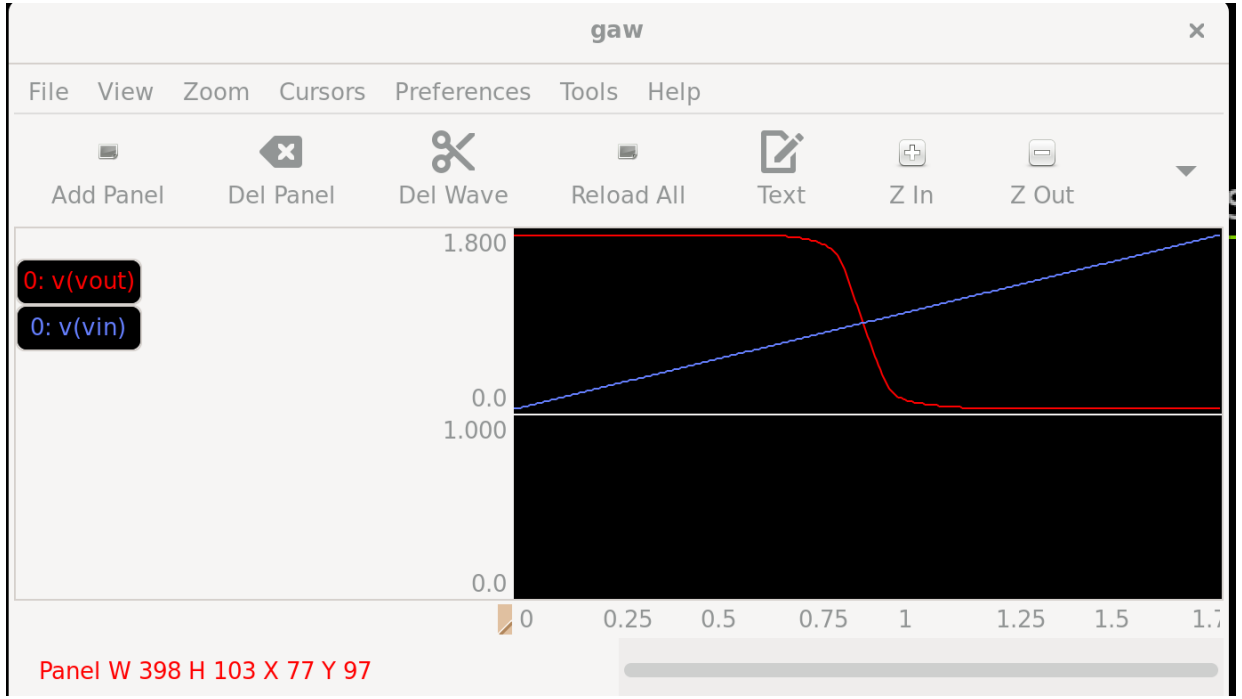
wrdata と gawによる波形表示

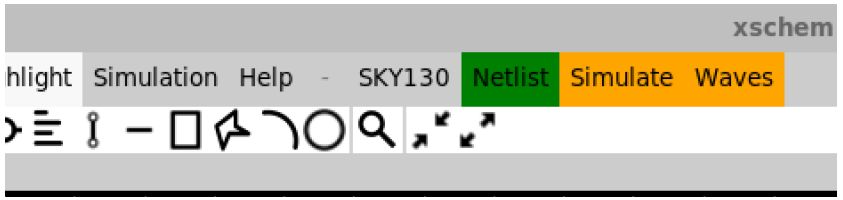
wrdataを使用するとテキスト出力ができる

0.00000000e+00	3.29999999e+00
5.00000000e-02	3.29999999e+00
1.00000000e-01	3.29999997e+00
1.50000000e-01	3.29999990e+00
2.00000000e-01	3.29999960e+00
2.50000000e-01	3.29999842e+00
3.00000000e-01	3.29999375e+00
3.50000000e-01	3.29997564e+00
4.00000000e-01	3.29990878e+00
4.50000000e-01	3.29968137e+00
5.00000000e-01	3.29900073e+00
5.50000000e-01	3.29728006e+00

.
.
.

writeでrawファイルを出力すると gaw で波形表示ができる





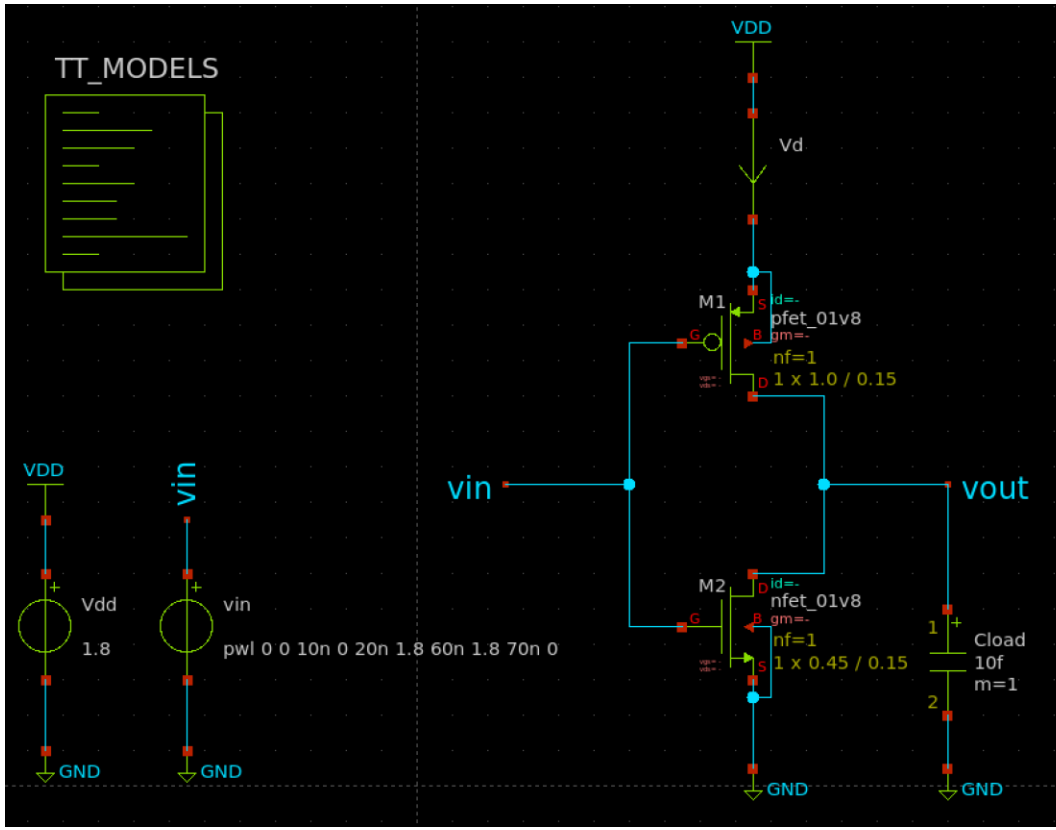
xschem メニューの
Wavesから起動できる

過渡応答、遅延時間

- 過渡応答から遅延時間を見たい場合はtran解析を用いる

- 出力段に負荷を設定しないと正しい過渡応答が見れない
  今回は入力段・出力段共に何も接続しない状態を見る

ベンチマーク例



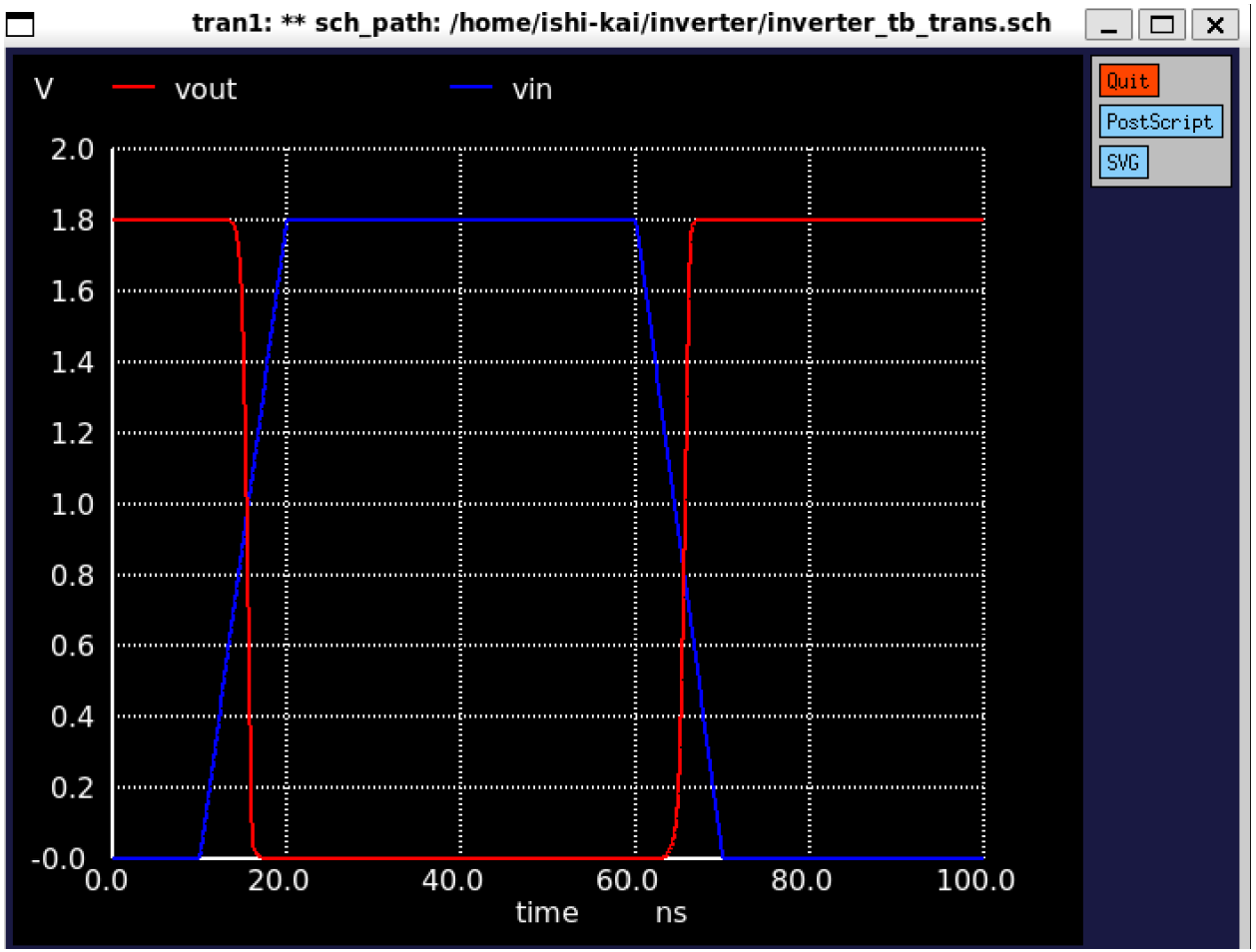
spice

```
.option savecurrent
.control
save all

* Tran analysis
tran 0.1n 100n
plot vout vin
plot i(vd)
wrdata ~/inverter_tb_tran.txt v(vout)
write inverter_tb_trans.raw
.endc
```

measure

```
.measure tran td_r trig v(vin) val=0.9 fall=1 targ v(vout) val=0.9 rise=1
.measure tran td_f trig v(vin) val=0.9 rise=1 targ v(vout) val=0.9 fall=1
.measure tran trise trig v(vout) val=0.4 rise=1 targ v(vout) val=1.4 rise=1
.measure tran tfall trig v(vout) val=1.4 fall=1 targ v(vout) val=0.4 fall=1
```

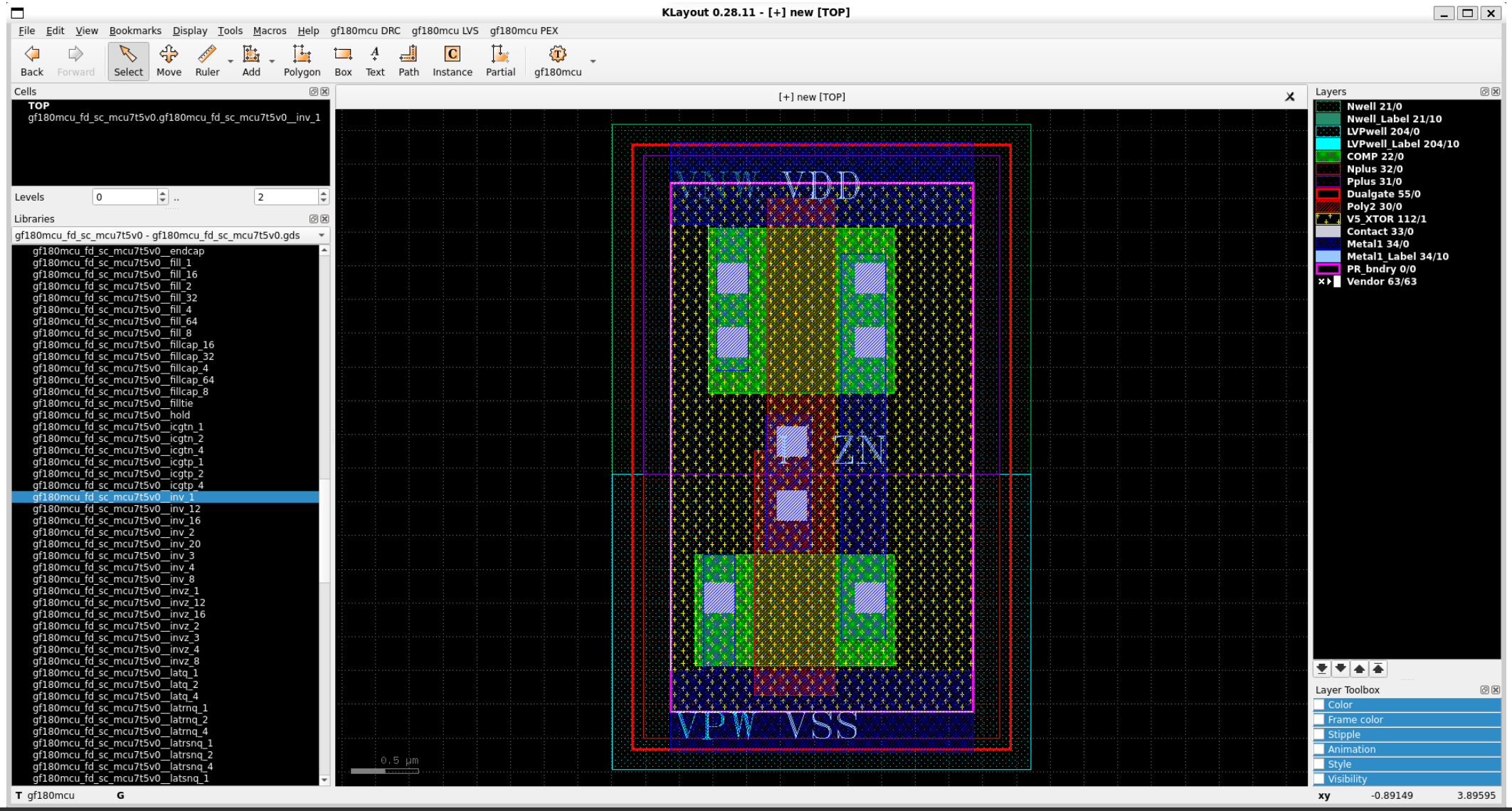
シミュレーション結果


tran1: ** sch_path: /home/ishi-kai/inverter/inverter_tb_trans.sch
V
vout
vin
2.0
1.8
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
-0.0
0.0
20.0
40.0
60.0
80.0
100.0
time
ns
Quit
PostScript
SVG

アナログLSIの設計フロー

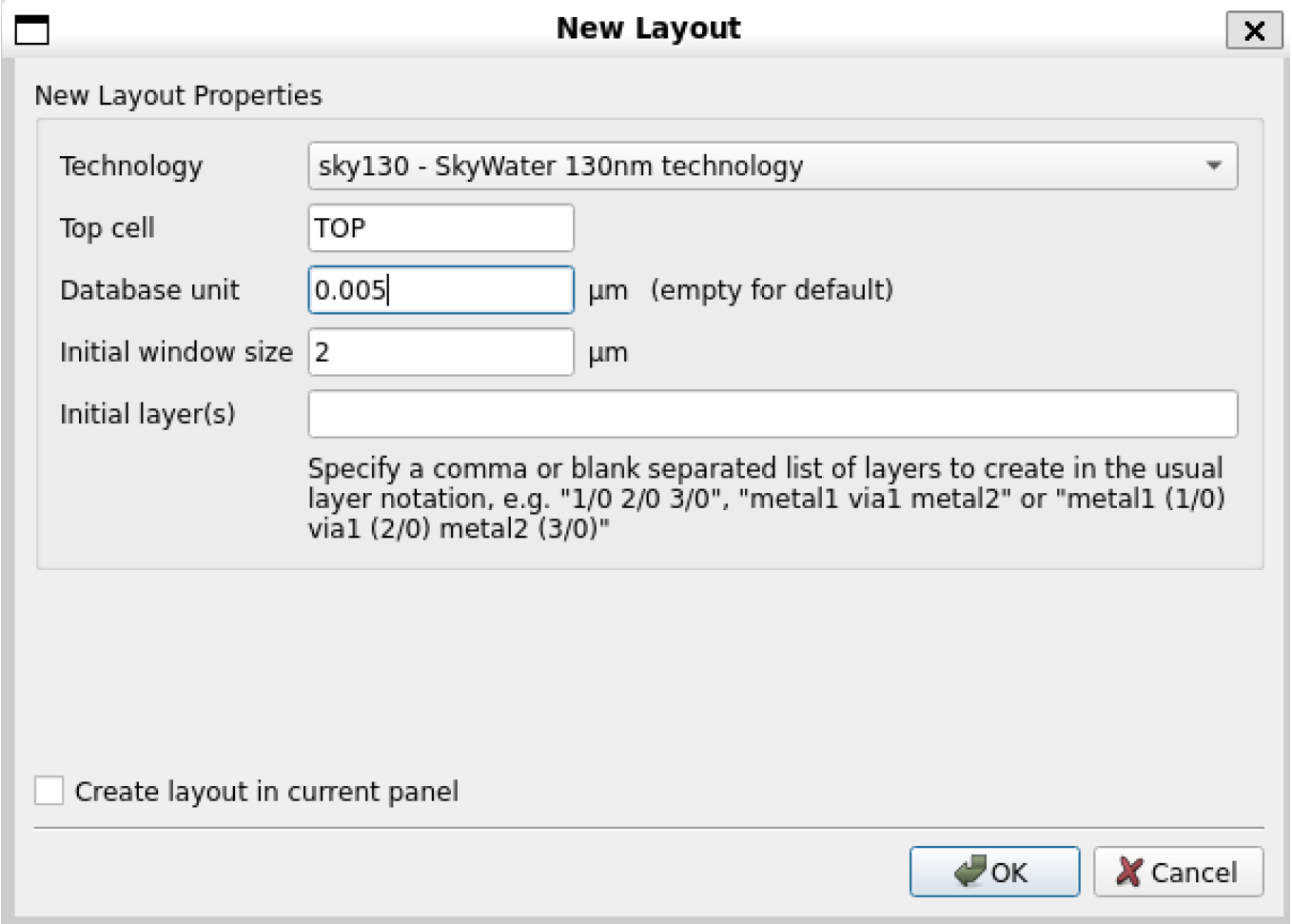
1. 回路図（ベンチマーク）を描く
2. シミュレーションをする
3. **回路図を基にレイアウトを描く**
4. レイアウトを検証する
5. レイアウトを基に寄生成分を考慮したシミュレーションをする
6. （フレームに載せる）

レイアウト画面


KLayout 0.28.11 - [+] new [TOP]
File Edit View Bookmarks Display Tools Macros Help gf180mcu DRC gf180mcu LVS gf180mcu PEX
Back Forward Select Move Ruler Add Polygon Box Text Path Instance Partial gf180mcu
Cells
TOP
gf180mcu_fd_sc_mcu7t5v0.gf180mcu_fd_sc_mcu7t5v0__inv_1
Levels 0 .. 2
Libraries
gf180mcu_fd_sc_mcu7t5v0 - gf180mcu_fd_sc_mcu7t5v0.gds
gf180mcu_fd_sc_mcu7t5v0__endcap
gf180mcu_fd_sc_mcu7t5v0__fill_1
gf180mcu_fd_sc_mcu7t5v0__fill_16
gf180mcu_fd_sc_mcu7t5v0__fill_2
gf180mcu_fd_sc_mcu7t5v0__fill_32
gf180mcu_fd_sc_mcu7t5v0__fill_4
gf180mcu_fd_sc_mcu7t5v0__fill_64
gf180mcu_fd_sc_mcu7t5v0__fill_8
gf180mcu_fd_sc_mcu7t5v0__fillcap_16
gf180mcu_fd_sc_mcu7t5v0__fillcap_32
gf180mcu_fd_sc_mcu7t5v0__fillcap_4
gf180mcu_fd_sc_mcu7t5v0__fillcap_64
gf180mcu_fd_sc_mcu7t5v0__fillcap_8
gf180mcu_fd_sc_mcu7t5v0__filltie
gf180mcu_fd_sc_mcu7t5v0__hold
gf180mcu_fd_sc_mcu7t5v0__icgtn_1
gf180mcu_fd_sc_mcu7t5v0__icgtn_2
gf180mcu_fd_sc_mcu7t5v0__icgtn_4
gf180mcu_fd_sc_mcu7t5v0__icgtp_1
gf180mcu_fd_sc_mcu7t5v0__icgtp_2
gf180mcu_fd_sc_mcu7t5v0__icgtp_4
gf180mcu_fd_sc_mcu7t5v0__inv_1
gf180mcu_fd_sc_mcu7t5v0__inv_12
gf180mcu_fd_sc_mcu7t5v0__inv_16
gf180mcu_fd_sc_mcu7t5v0__inv_2
gf180mcu_fd_sc_mcu7t5v0__inv_20
gf180mcu_fd_sc_mcu7t5v0__inv_3
gf180mcu_fd_sc_mcu7t5v0__inv_4
gf180mcu_fd_sc_mcu7t5v0__inv_8
gf180mcu_fd_sc_mcu7t5v0__invz_1
gf180mcu_fd_sc_mcu7t5v0__invz_12
gf180mcu_fd_sc_mcu7t5v0__invz_16
gf180mcu_fd_sc_mcu7t5v0__invz_2
gf180mcu_fd_sc_mcu7t5v0__invz_3
gf180mcu_fd_sc_mcu7t5v0__invz_4
gf180mcu_fd_sc_mcu7t5v0__invz_8
gf180mcu_fd_sc_mcu7t5v0__latq_1
gf180mcu_fd_sc_mcu7t5v0__latq_2
gf180mcu_fd_sc_mcu7t5v0__latq_4
gf180mcu_fd_sc_mcu7t5v0__latrnq_1
gf180mcu_fd_sc_mcu7t5v0__latrnq_2
gf180mcu_fd_sc_mcu7t5v0__latrnq_4
gf180mcu_fd_sc_mcu7t5v0__latrsnq_1
gf180mcu_fd_sc_mcu7t5v0__latrsnq_2
gf180mcu_fd_sc_mcu7t5v0__latrsnq_4
gf180mcu_fd_sc_mcu7t5v0__latsnq_1
[+] new [TOP]
VDD
ZN
VPW VSS
0.5 µm
Layers
Nwell 21/0
Nwell_Label 21/10
LVPwell 204/0
LVPwell_Label 204/10
COMP 22/0
Nplus 32/0
Pplus 31/0
Dualgate 55/0
Poly2 30/0
V5_XTOR 112/1
Contact 33/0
Metal1 34/0
Metal1_Label 34/10
PR_bndry 0/0
Vendor 63/63
Layer Toolbox
Color
Frame color
Stipple
Animation
Style
Visibility
T gf180mcu G
xy -0.89149 3.89595

レイアウトを描く上での注意

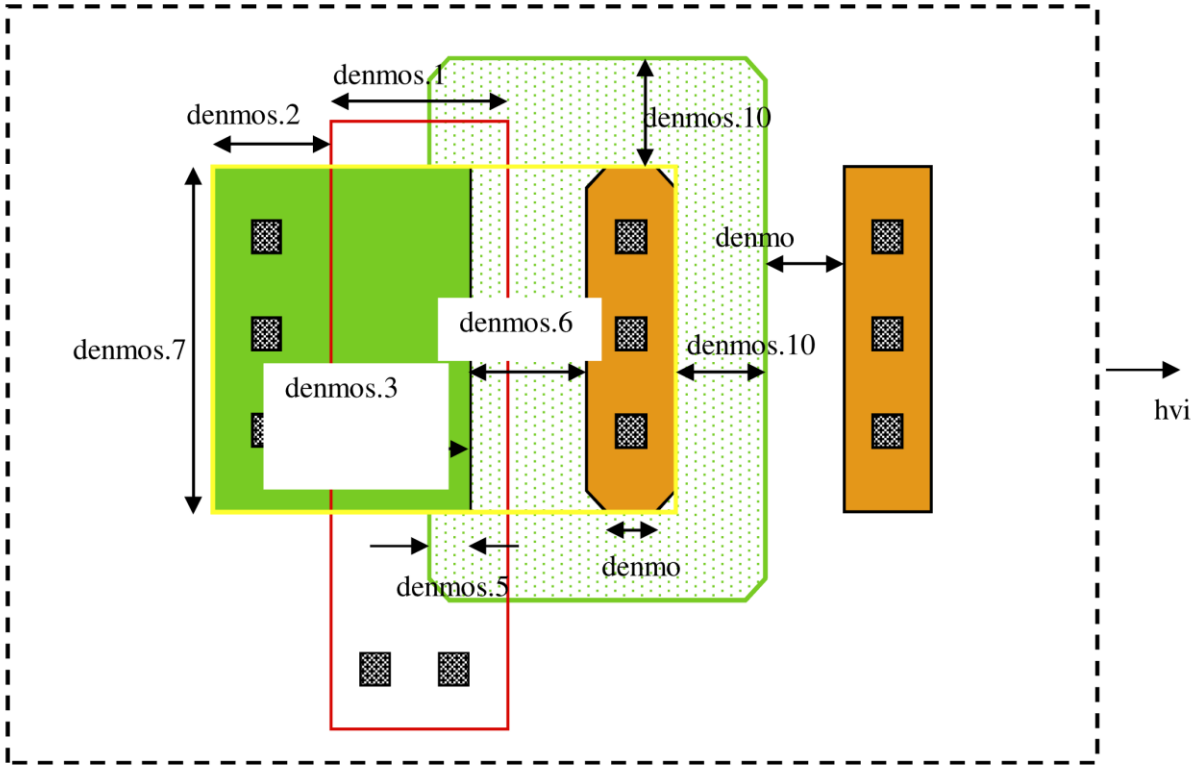
- **Database unit を 0.005にする**



デザインルールの場所

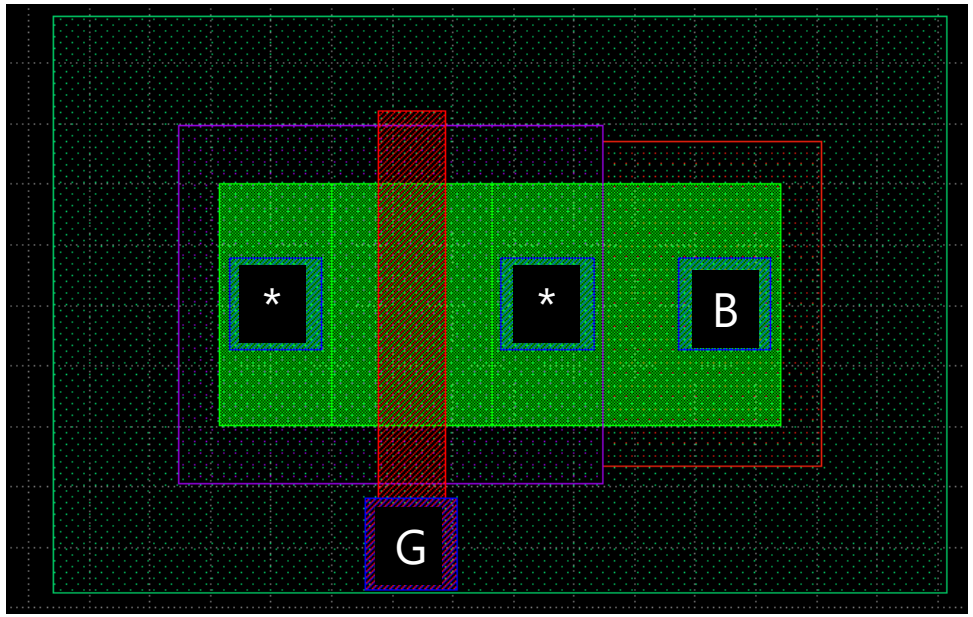
- 必ずデザインルールを守る

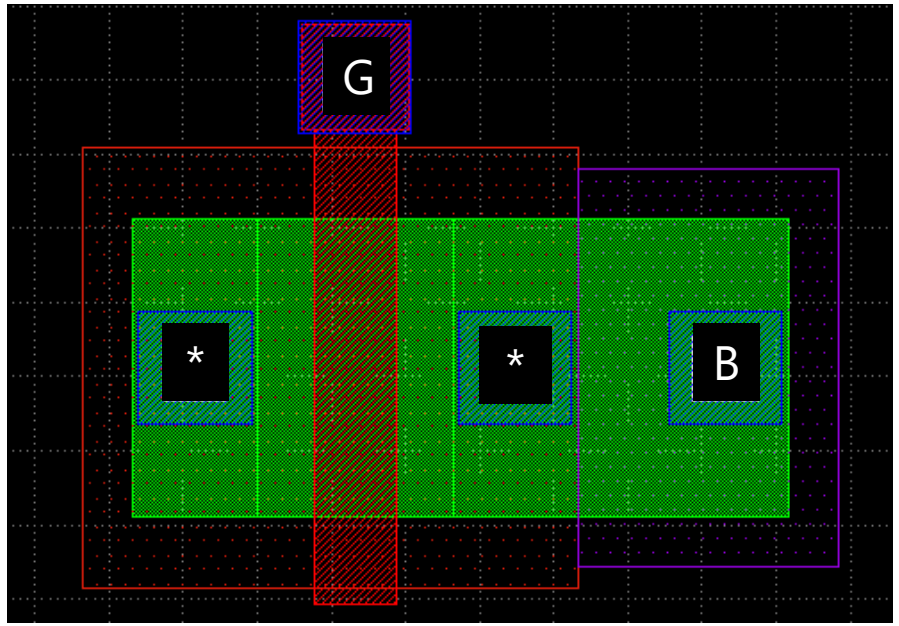
https://skywater-pdk.readthedocs.io/en/main/rules/periphery.html#x



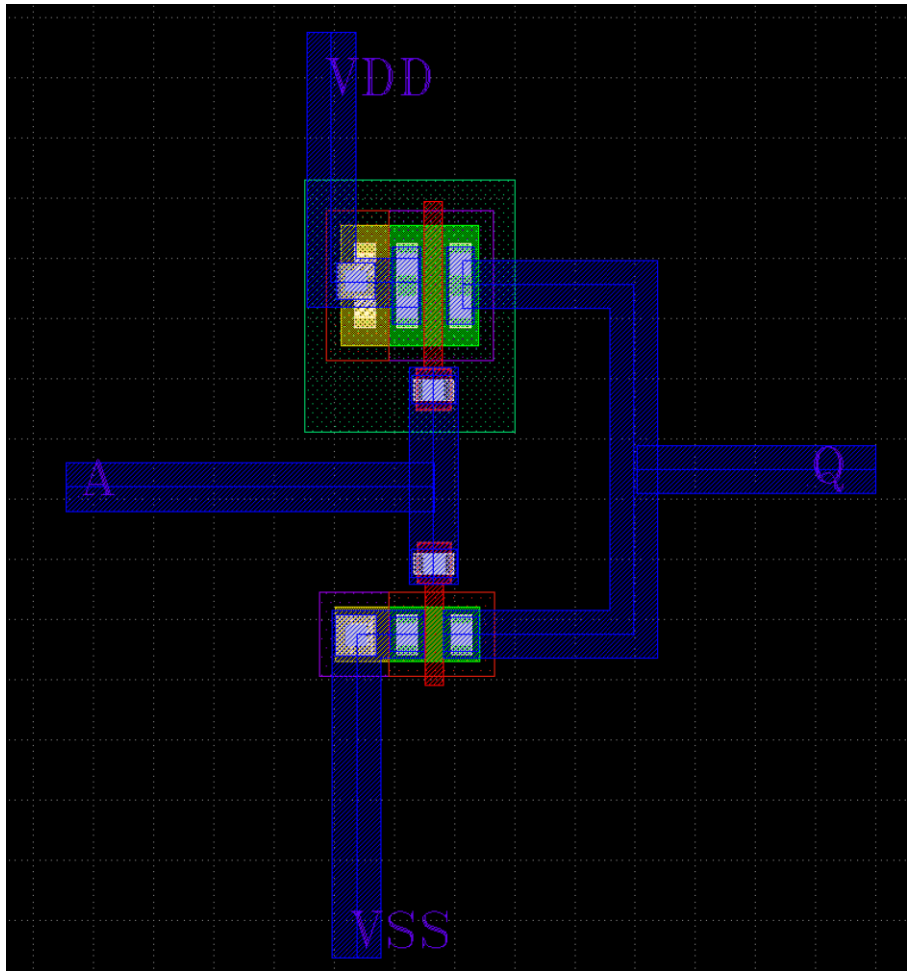
タップ付　PFET と NFET

- [*] どちらをドレインにしてどちらをソースにするかは任意。





サンプルレイアウト



アナログLSIの設計フロー

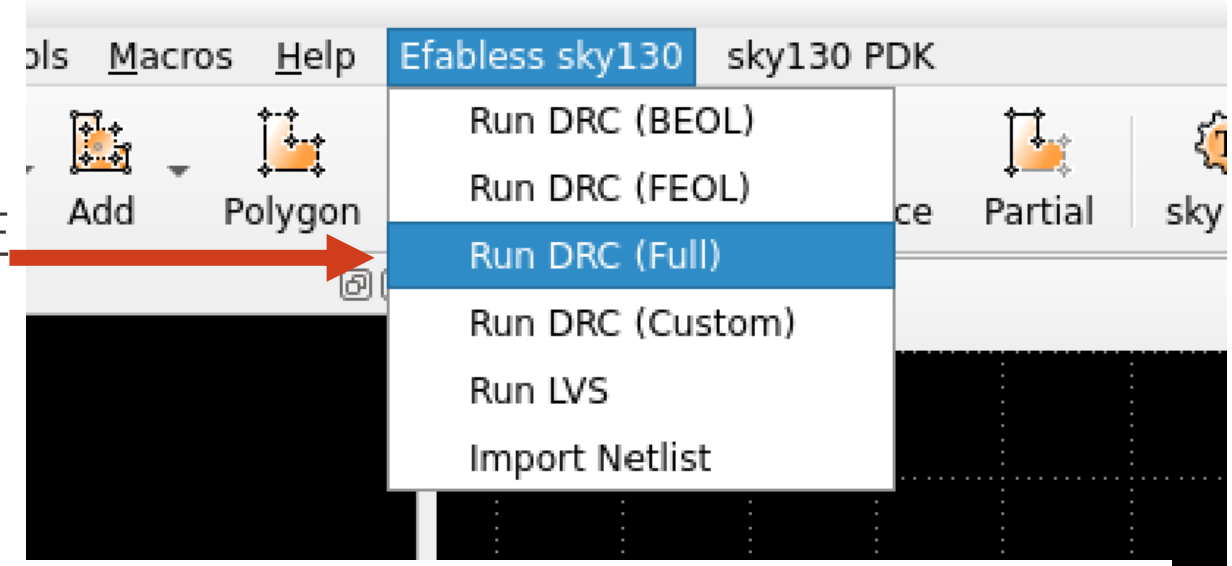
1. 回路図（ベンチマーク）を描く
2. シミュレーションをする
3. 回路図を基にレイアウトを描く
4. **レイアウトを検証する**
5. レイアウトを基に寄生成分を考慮したシミュレーションをする

6. （フレームに載せる）

3zki / wsl_gf180mcu で利用できる レイアウト検証

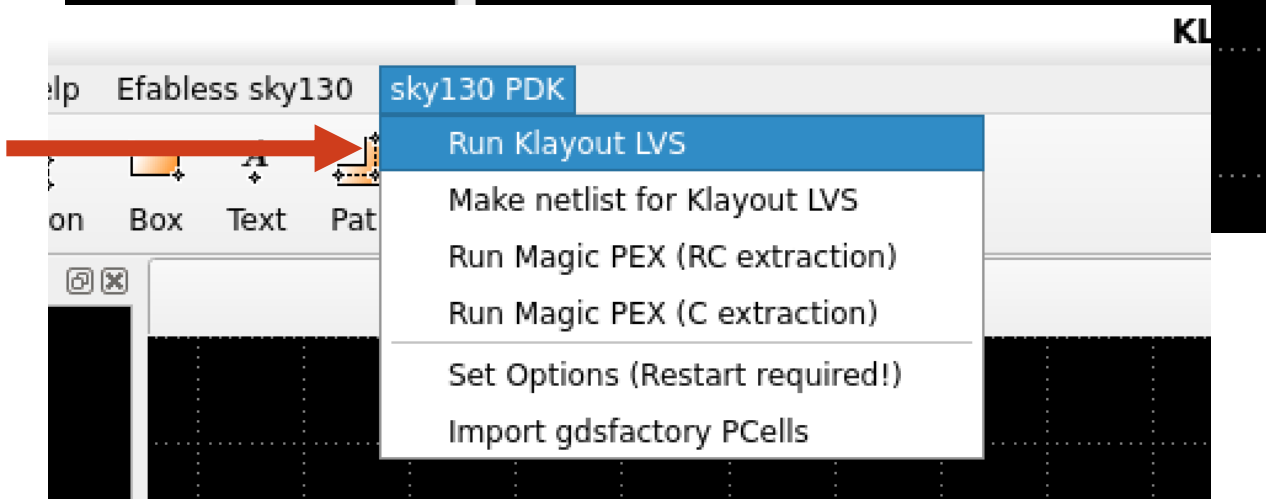
Design Rules Check (DRC)

- 指定されたデザインルールから違反していないか検証



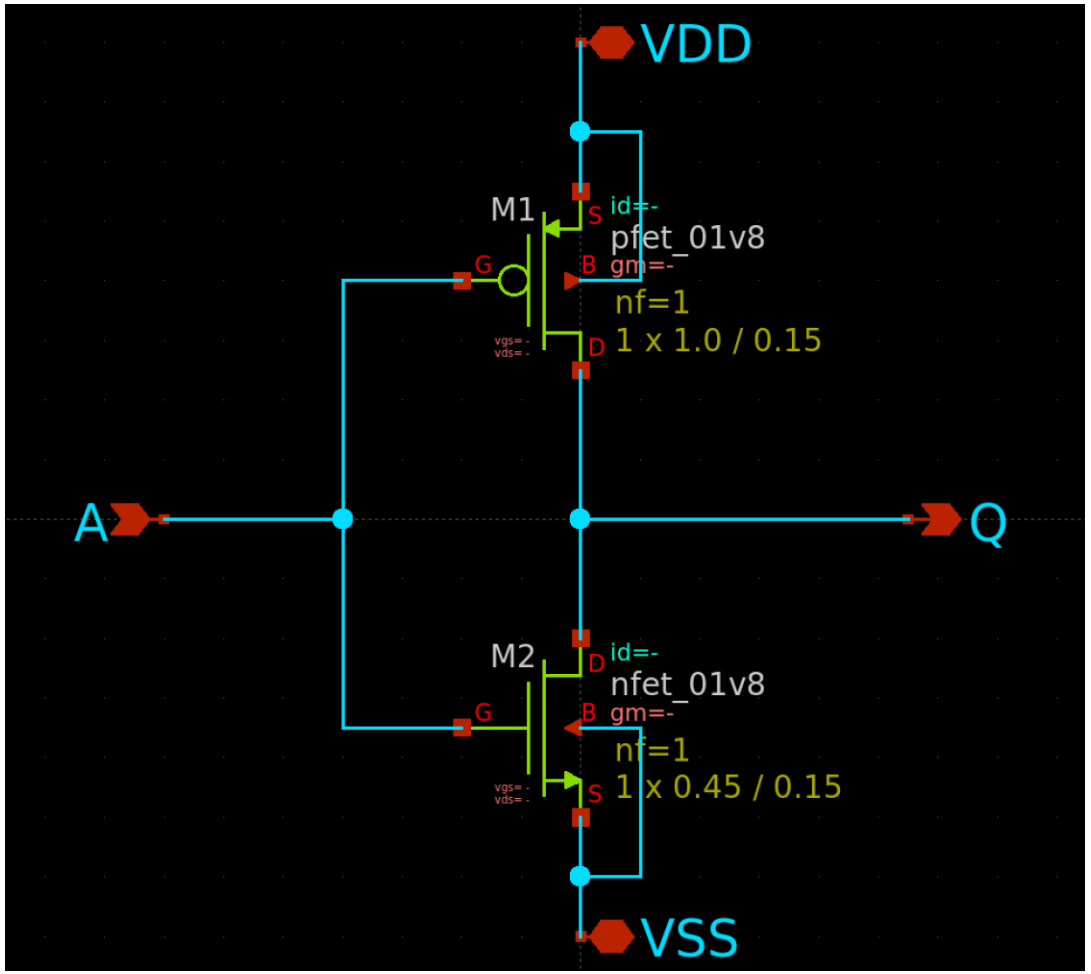
Layout Versus Schematic (LVS)

- レイアウトが回路図通りに描けているか検証



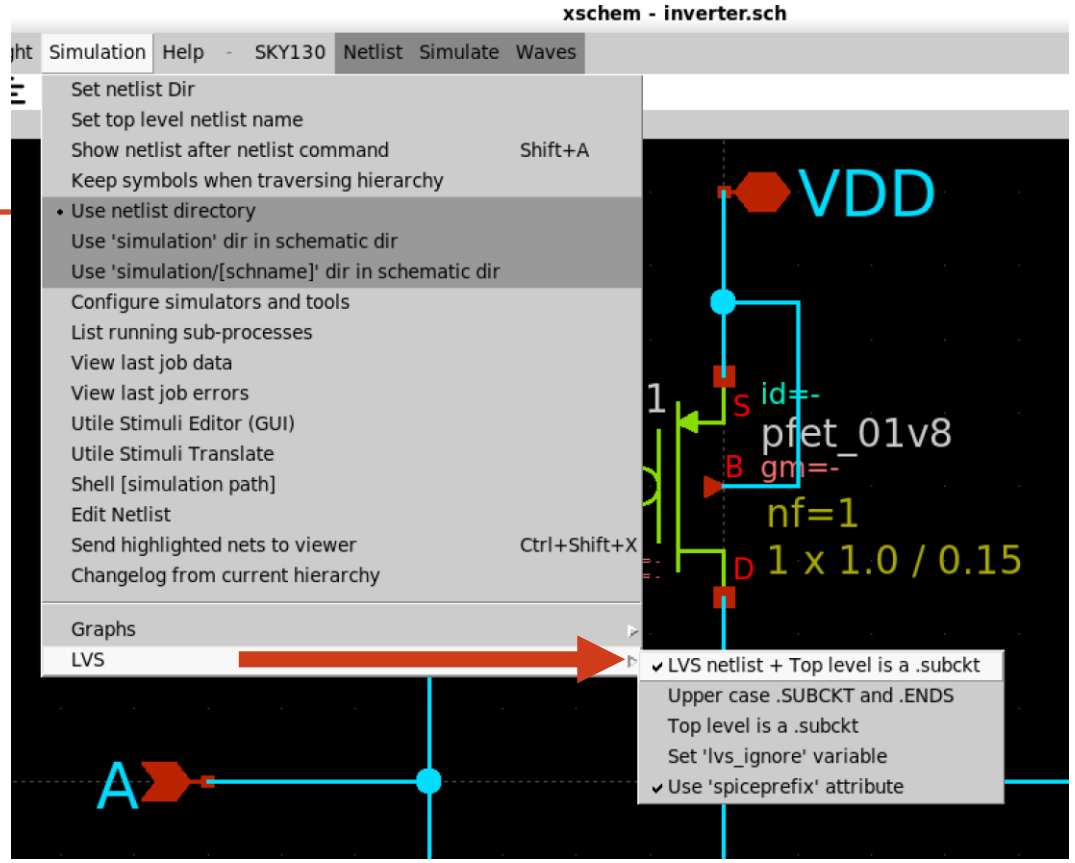
LVS用ネットリストの出力方法 (1/2)

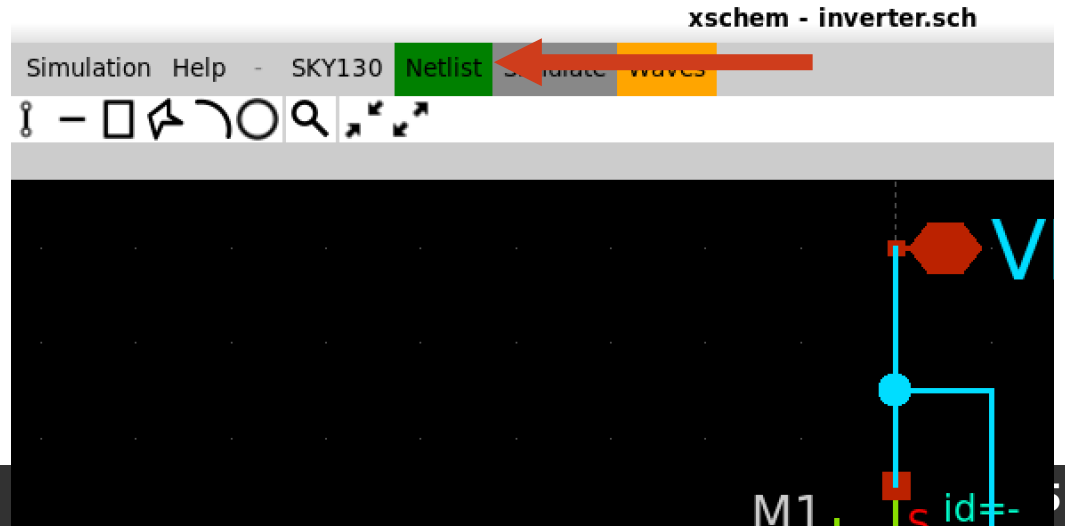
- xschem上であらかじめLVS用の回路図を作成しておく。
- ピンはipin, iopin, opin のみを使用する。
  - vdd.symやgnd.symを使用するとうまく通らない。
- ファイルを同じ名前にする 例: nand.sch, nand.gds



LVS用ネットリストの出力方法（1/2）

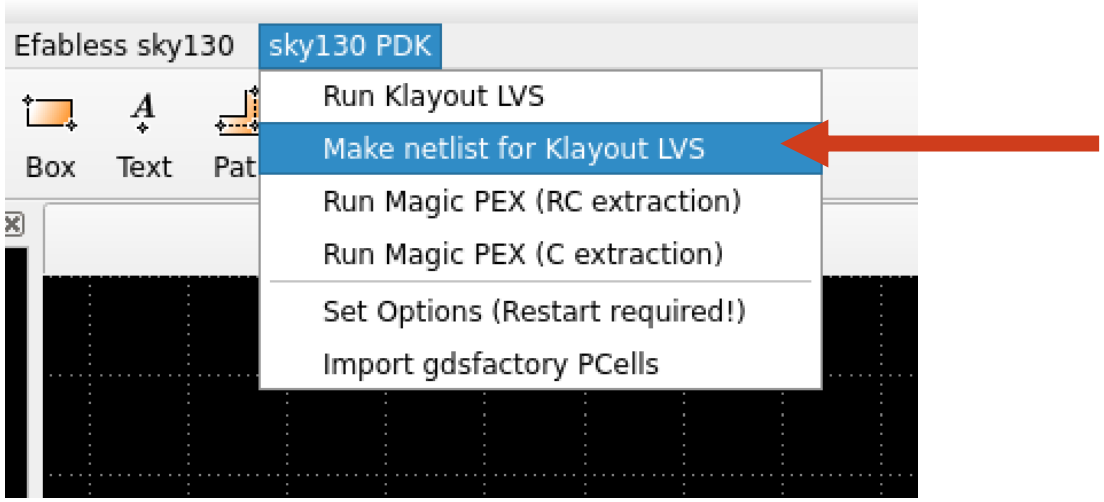
1. xschem上で**LVS用**のネットリストを出力する。
    1. Simulation -> LVS -> LVS netlist + Top level is a.subckt にチェックを入れる
    2. Netlistを押す
    3. 終わったら閉じて良い
    4. デフォルトの生成場所は ~/.xschem/simulations/





LVS用ネットリストの出力方法（2/2）

1. Klayout上で上記のLVS用ネットリストをKlayout用に変換する
    1. Make netlist for Klayout LVSを押す
2. LVSを実行する
    1. Run Klayout LVSを押す



アナログLSIの設計フロー

1. 回路図（ベンチマーク）を描く
2. シミュレーションをする
3. 回路図を基にレイアウトを描く
4. レイアウトを検証する
5. **レイアウトを基に寄生成分を考慮したシミュレーションをする**
6. （フレームに載せる）

レイアウト検証：PEX, Parasitic Extraction

RC抽出

- 寄生抵抗と寄生容量を含むネットリストを抽出

Magic-PEX

File Edit View Search Terminal Help

```
exttospice finished.
* NGSPICE file created from TOP.ext - technology: gf180mcuC

.subckt TOP A Q VDD GND
X0 Q A.t0 VDD.t1 VDD.t0 pfet_03v3 ad=1.3p pd=5.3u as=1.3p ps=5.3u w=2u l=0.28u
X1 Q A.t1 GND.t1 GND.t0 nfet_03v3 ad=0.61p pd=3.22u as=0.61p ps=3.22u w=1u l=0.2
8u
R0 A.n0 A.t1 60.6324
R1 A.n0 A.t0 50.9859
R2 A A.n0 4.00168
R3 VDD.n0 VDD.t0 909.662
R4 VDD.n0 VDD.t1 4.56542
R5 VDD VDD.n0 0.0465872
R6 Q Q.n1 6.1565
R7 Q Q.n0 4.60529
R8 GND.n0 GND.t0 2545.79
R9 GND.n0 GND.t1 6.08384
R10 GND GND.n0 0.0514016
C0 VDD Q 0.146f
C1 A Q 0.0875f
C2 A VDD 0.139f
.ends
```

C抽出

- 寄生容量のみを含むネットリストを抽出

Magic-PEX

File Edit View Search Terminal Help

```
s fillblock lvstext obscomment
Scaled tech values by 10 / 1 to match internal grid scaling
Loading gf180mcuC Device Generator Menu ...
Loading "/home/user/.klayout/macros/gf180mcu_magic_pex.tcl" from command line.
Warning: Calma reading is not undoable!  I hope that's OK.
Library written using GDS-II Release 6.0
Library name: LIB
Reading "TOP".
CIF file read warning: CIF style import: units rescaled by factor of 5 / 1
Extracting TOP into TOP.ext:
exttosim finished.
exttospice finished.
exttospice finished.
* NGSPICE file created from TOP.ext - technology: gf180mcuC

.subckt TOP A Q VDD GND
X0 Q A VDD VDD pfet_03v3 ad=1.3p pd=5.3u as=1.3p ps=5.3u w=2u l=0.28u
X1 Q A GND GND nfet_03v3 ad=0.61p pd=3.22u as=0.61p ps=3.22u w=1u l=0.28u
C0 A Q 0.0875f
C1 VDD Q 0.146f
C2 A VDD 0.139f
.ends
```

PEX前の下準備：Flatten

1. 各ピンにラベルを付ける
    1. metN.pinレイヤーを使用すること
2. コピーを保存する

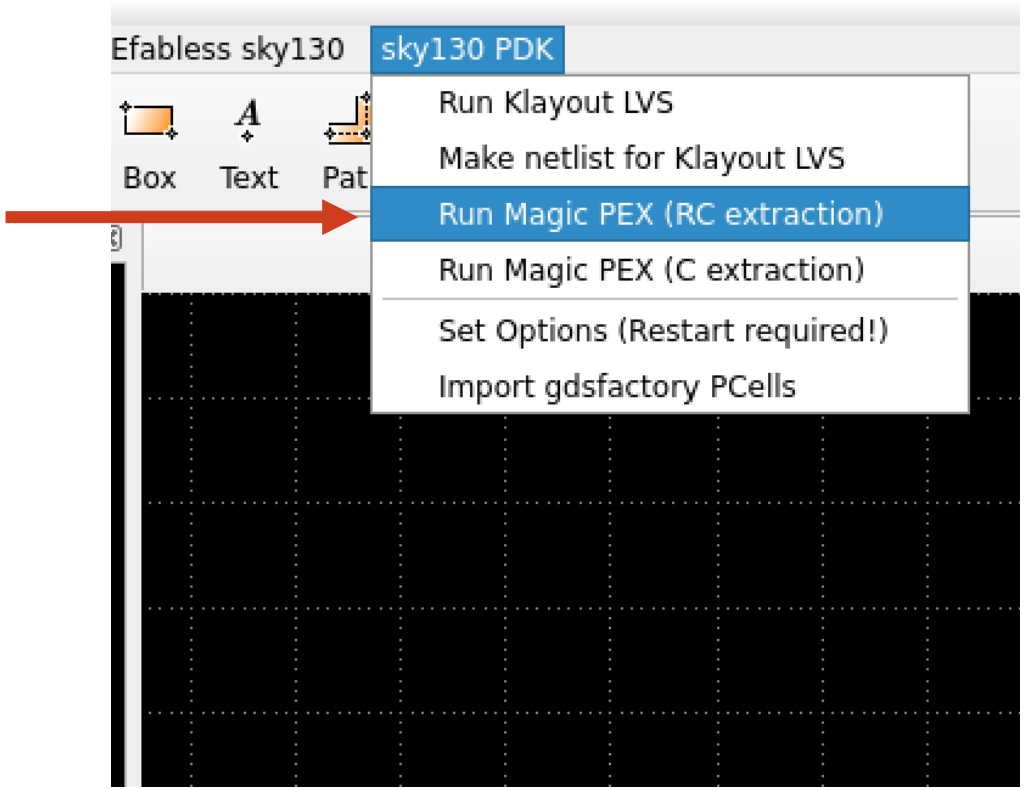
- ファイル名で大文字を使うとエラーが出てしまう（不具合）
    - 拡張子がデフォルトだと.GDSなので.gdsに直す

3. ドラッグでレイアウト全体を選択する
4. Edit > Flatten Cell > Flatten Instances を All hierarchy levels で実行
5. Cells が TOP のみになっていることを確認して Save

寄生抽出

Magic PEX を用いて寄生容量抽出を行う

- Run Magic PEX (C extraction) を実行



実行すると寄生容量付きネットリストが生成される

- ファイル名：{レイアウト名}_pex_extracted.spice
  - デフォルトではTOP_pex_extracted.spice

寄生容量を考慮したベンチマークに変更する

ポストシミュレーションの際は以下 2 点の変更が必要

1. 外部ネットリストへのパス
2. 外部ネットリスト用のシンボル

外部ネットリストへのパスを通す

- シミュレーションスクリプトに.includeでパスを通す
  - 絶対パスにする必要がある点に注意！
    - レイアウトデータと同じ階層に [レイアウト名]_pex_extracted.spice が生成される

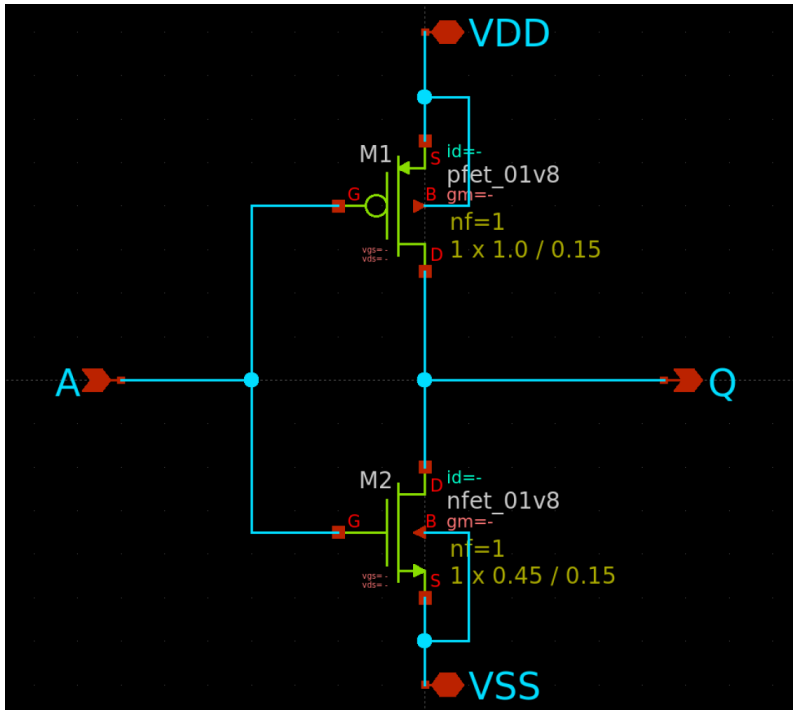
spice

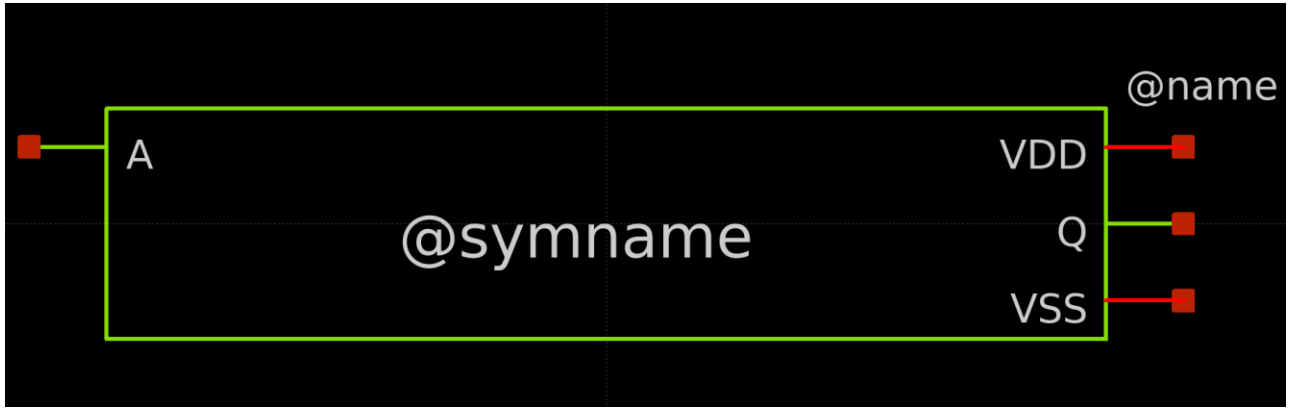
```
.option savecurrent
.include ~/TOP_pex_extracted.spice
.control
save all

* DC analysis (I/O curve)
dc vin 0 1.8 0.01
plot vout1 vout2 vin
plot i(vd)
wrdata ~/inverter_pex_tb.txt v(vout)
write inverter_pex_tb.raw
.endc
```

外部ネットリスト用のシンボル作成

LVSで使用した回路図からシンボルを作成する

- メニュー Symbol > Make symbol from schematic





このままだとシンボルはLVSで使用した回路図を参照するため
ネットリストを参照するように改変する

外部ネットリスト用のシンボル作成　（基本形）

シンボルのプロパティを開いてこんな感じの中身にする　（Qキーでプロパティを開く）

type=primitive

format="@name [ネットリスト内のピン] @prefix"

template="name=x1 prefix=[subcktの名前]"

extra="prefix"

highlight=true

詳しくは http://repo.hu/projects/xschem/xschem_man/symbol_property_syntax.html を参照

外部ネットリスト用のシンボル作成例　（インバータ）

type=primitive

format="@name @@Q @@A @@VSS @@VDD @prefix“

template="name=x1 prefix=TOP“

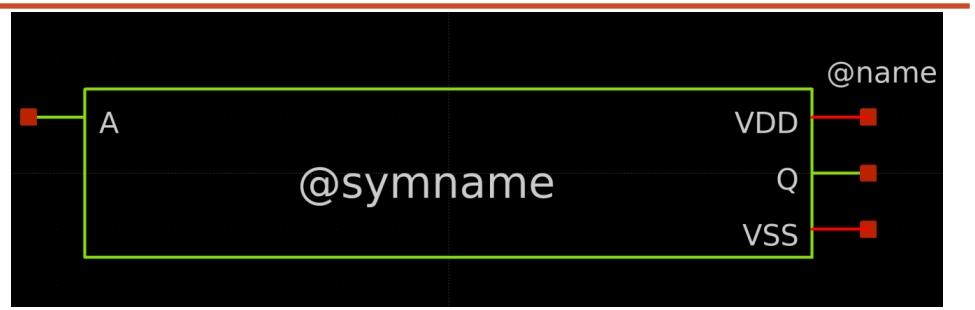
extra="prefix“

highlight=true

```
.subckt TOP Q A VSS VDD
X0 Q A VDD VDD sky130_fd_p
X1 Q A VSS VSS sky130_fd_p
C0 A VDD 0.15271f
C1 Q A 0.08874f
C2 Q VDD 0.1357f
C3 Q VSS 0.71424f
C4 A VSS 0.62185f
C5 VDD VSS 0.68683f
.ends
```

抽出したネットリストのピンと同じ並びで記述する

外部ネットリスト用のシンボル作成例　（インバータ）



type=primitive

format="@name @@Q @@A @@VSS @@VDD @prefix“

template="name=x1 prefix=TOP“

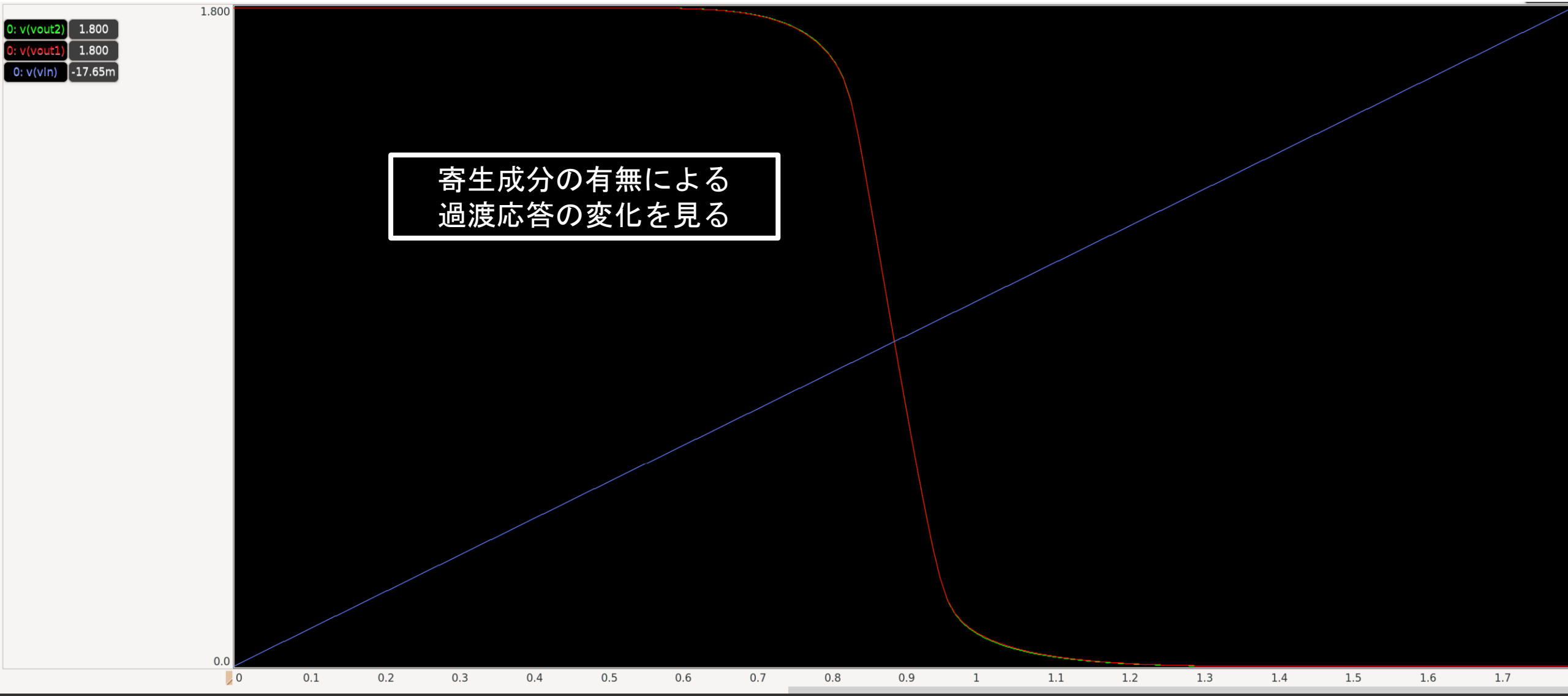
extra="prefix“

highlight=true

シンボルから出ているピンには“@@”のprefixをつける

これで一応完成

ベンチマーク例


0: v(vout2) 1.800
0: v(vout1) 1.800
0: v(vin) -17.65m
1.800
0.0
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1
1.1
1.2
1.3
1.4
1.5
1.6
1.7
寄生成分の有無による
過渡応答の変化を見る

外部ネットリスト用のシンボル作成　（応用形）

シンボルのプロパティを開いてこんな感じの中身にする　（Qキーでプロパティを開く）

type=primitive

format="@name [ネットリスト内のピン] @prefix"

template="name=x1 [プロパティでネットを指定するピン] prefix=TOP"

extra=" [プロパティでネットを指定するピン] prefix"

highlight=true

詳しくは http://repo.hu/projects/xschem/xschem_man/symbol_property_syntax.html を参照

外部ネットリスト用のシンボル作成（応用例）

VDDとGNDのピンをシンボルから削除してプロパティで記述してみる

type=primitive

format="@name @@A @@Y @VDD @GND @prefix“

template="name=x1 VDD=VDD GND=GND prefix=TOP“

extra=“VDD GND prefix“

highlight=true

プロパティでピンのネットを指定する場合は”@”のprefixを付けて、extraにピン、templateにデフォルトの値を追加する

外部ネットリスト用のシンボル作成（応用例）

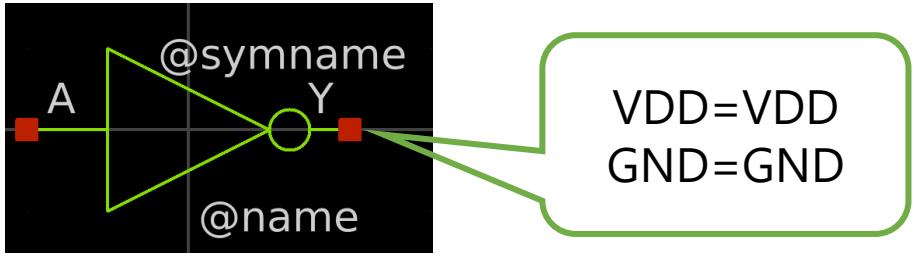
VDDとGNDのピンをシンボルから削除してプロパティで記述してみる

type=primitive

format="@name @@A @@Y @VDD @GND @prefix“

template="name=x1 VDD=VDD GND=GND prefix=TOP“

extra=“VDD GND prefix“

highlight=true

ベンチマークが
見やすくなる